Quinnipiac University

School of Computing and Engineering
Department of Computer Science

Hybrid Quantum-AI Models for Cybersecurity on Cloud Platforms: Accuracy, Latency, and Energy

HERO -- A Heterogeneous Cloud Framework for Cost-Energy-Latency Trade-offs in Quantum-Classical Workloads

A Master's Thesis
submitted in partial fulfillment of the requirements for the degree of
Master of Science in Computer Science

by

Nirmit Dagli

Thesis Advisor: Prof. Taskin
Thesis Committee: Prof. Kruti, Prof. Lin

Hamden, Connecticut, USA
May 2026
Academic Year 2025/2026

Declaration of Authorship

I, Nirmit Dagli, hereby declare that this thesis titled "Hybrid Quantum-AI Models for Cybersecurity on Cloud Platforms: Accuracy, Latency, and Energy" and the work presented in it are my own. I confirm that:

1. This work was conducted wholly while in candidature for a Master's degree at Quinnipiac University.
2. Where any part of this thesis has previously been submitted for a degree or any other qualification at this university or any other institution, this has been clearly stated.
3. Where I have consulted the published work of others, this is always clearly attributed using IEEE-style numerical citations.
4. Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
5. I have acknowledged all main sources of help.
6. All quantum simulation, classical baseline implementations, energy and cost measurements, and statistical analyses presented in this thesis were performed by the author using publicly available software libraries (Qiskit, scikit-learn, NumPy, SciPy, matplotlib).
7. The HERO framework, including its workload-aware tier allocation rule, Pareto selection logic, and cloud cost model, is the original contribution of the author.

Signed: _____

Name: Nirmit Dagli

Date: _____

Acknowledgements

I would like to express my deepest gratitude to my thesis advisor, Prof. Taskin, for the continuous guidance, technical insight, and patient supervision throughout the duration of this Master's thesis. The discussions on heterogeneous computing, quantum algorithms, and systematic measurement methodology have shaped both the framing and the depth of this work.

I am equally grateful to Prof. Kruti and Prof. Lin, members of my thesis committee, for their critical reading, thoughtful questions, and constructive feedback. Their input on cloud-platform context and on the cybersecurity and molecular-simulation domains substantially strengthened the contributions presented here.

I thank the Department of Computer Science at Quinnipiac University for providing an environment in which interdisciplinary research at the intersection of quantum computing, artificial intelligence, and cloud systems is encouraged. I acknowledge the open-source community -- particularly the maintainers of Qiskit, scikit-learn, NumPy, and matplotlib -- whose tools made this research practically possible.

I am deeply indebted to my family, whose support and encouragement made this Master's degree possible. I dedicate this thesis to them.

Nirmit Dagli
Hamden, Connecticut, USA, May 2026

Abstract

This thesis presents the Heterogeneous Energy-aware Runtime Orchestrator (HERO), a measurement framework for hybrid Quantum-Classical Artificial Intelligence (AI) workloads deployed on cloud platforms that expose heterogeneous compute tiers: Central Processing Unit (CPU), Graphics Processing Unit (GPU), and Quantum Processing Unit (QPU). The central research question is: given a workload, how should one allocate it across these cloud tiers to optimise the joint objective of cost, energy, and latency without sacrificing accuracy?

We address this question through two contrasting case studies. The first is cybersecurity intrusion detection on the Knowledge Discovery and Data Mining (KDD) Cup 99 dataset, where a Quantum Support Vector Machine (QSVM) is benchmarked against four classical classifiers (Random Forest, Gradient Boosting, K-Nearest Neighbors, and Support Vector Machine). The second is molecular ground-state estimation of the H2 molecule via the Variational Quantum Eigensolver (VQE), benchmarked against exact NumPy diagonalisation. Each experiment is repeated 30 times for statistical validation, and per-task accuracy, latency, energy, and 2026 cloud cost are recorded.

Our results show two opposite verdicts. For cybersecurity, classical Random Forest achieves an F1 score of 0.998 against QSVM at 0.667, while QSVM costs approximately 10,000 times more energy and roughly two million US dollars per million classifications versus less than two dollars for classical Support Vector Machine. For molecular simulation, exact diagonalisation trivially solves H2 today, yet the classical state-vector approach scales as O(2^n) and becomes infeasible beyond approximately fifty qubits, while VQE scales polynomially.

From these measurements we derive a workload-aware tier allocation rule (Algorithm 1), formulate a Pareto-frontier selection method over the joint cost-energy-latency-accuracy space, and project the cross-over feature dimension at which quantum kernel methods become cost-competitive. The framework, the simulator, and all measurement scripts are released as open source so the community can re-measure as quantum hardware improves.

Keywords:

Hybrid Quantum-Classical Computing; Cloud Platforms; Energy Measurement; Cost-Energy-Latency Optimisation; Quantum Machine Learning; Intrusion Detection; Molecular Simulation; Variational Quantum Eigensolver; Quantum Support Vector Machine; Noisy Intermediate-Scale Quantum.

Table of Contents

Right-click and select 'Update Field' in Word

List of Figures

List of Tables

Right-click and select 'Update Field' in Word

List of Abbreviations and Symbols

AI	Artificial Intelligence
API	Application Programming Interface
CICIDS2017	Canadian Institute for Cybersecurity Intrusion Detection System 2017 dataset
COBYLA	Constrained Optimization BY Linear Approximation
CPU	Central Processing Unit
DAG	Directed Acyclic Graph
DoS	Denial of Service
F1	F1 score (harmonic mean of precision and recall)
FT	Fault-Tolerant (quantum hardware)
GBM	Gradient Boosting Machine
GPU	Graphics Processing Unit
HERO	Heterogeneous Energy-aware Runtime Orchestrator (this work)
HPC	High-Performance Computing
IDS	Intrusion Detection System
IoT	Internet of Things
KDD	Knowledge Discovery and Data Mining
KNN	K-Nearest Neighbors
ML	Machine Learning
N-BaIoT	Network-based Botnet attacks on IoT devices (dataset)
NISQ	Noisy Intermediate-Scale Quantum
NN	Neural Network
NSL-KDD	A refined version of the KDD Cup 99 dataset
NVML	NVIDIA Management Library
PCA	Principal Component Analysis
PUE	Power Usage Effectiveness
QAOA	Quantum Approximate Optimization Algorithm
QC	Quantum Computing
QML	Quantum Machine Learning
QPU	Quantum Processing Unit
QSVM	Quantum Support Vector Machine
RAPL	Running Average Power Limit
RBF	Radial Basis Function
RF	Random Forest
R2L	Remote-to-Local (attack class)
SLA	Service Level Agreement
SVM	Support Vector Machine
TDP	Thermal Design Power
U2R	User-to-Root (attack class)
VQE	Variational Quantum Eigensolver
ZZ	Pauli-Z tensor Pauli-Z product (entangling feature map)

Chapter 1 — Introduction

1.1 Motivation

Network traffic and the intrusion-detection workloads that police it have grown to a scale that defines the modern cloud. The KDD Cup 99 benchmark, although now historical in its specific feature set, remains a useful reference point precisely because it captures the shape of the problem: hundreds of thousands of connection records, tens of features, severely imbalanced attack classes, and a hard real-time requirement that the underlying cloud infrastructure be able to score new flows in milliseconds [26]. Real production intrusion-detection systems in 2026 routinely operate on billions of events per day across geographically distributed datacentres, and every new method is judged not only by its detection accuracy but by whether it fits inside the cloud operator's energy and cost envelope.

In parallel with this growth in classical workloads, the past five years have seen the emergence of cloud-accessible quantum hardware as a genuinely usable resource. IBM Quantum offers superconducting transmon devices through its Qiskit Runtime service [25]; Amazon Web Services exposes superconducting, trapped-ion, and neutral-atom back-ends through AWS Braket; and Microsoft Azure Quantum federates several hardware providers behind a common application programming interface. A researcher with a credit card and a Python notebook can today submit a circuit to a 100-qubit superconducting device and receive results within minutes. This degree of access is historically unprecedented and has provoked a wave of speculative claims about the imminent transformation of every computational domain by quantum methods.

Cybersecurity has been a particular focus of this speculation. The popular narrative -- repeated in vendor whitepapers, conference keynotes, and an increasing number of academic surveys -- is that quantum computing will revolutionise intrusion detection, malware classification, and threat intelligence. The reasoning is rarely made precise. It is sometimes suggested that quantum kernel methods will provide an exponential separation; sometimes that quantum machine learning will simply train faster; sometimes, more vaguely, that quantum hardware will provide a generic accuracy boost. What is almost never offered is a quantitative measurement of accuracy, latency, energy, and cost on a representative cybersecurity dataset, performed with statistical rigour, on hardware (or a faithful simulator) that one can actually rent today.

At the same time, energy and cost have moved from being secondary considerations to first-class concerns for cloud workloads. Modern hyperscale datacentres are characterised by their Power Usage Effectiveness (PUE) -- the ratio of total facility energy to IT energy -- and the leading providers compete to bring this number below 1.2. Operators publish carbon dashboards; regulators in the European Union and several US states are beginning to require energy disclosure for AI training runs [23]. On the quantum side, per-second pricing for quantum processing units (QPUs) is extraordinarily high relative to classical compute -- a single shot on a leading-edge superconducting device costs in the order of a tenth of a US cent, and a typical kernel-evaluation routine requires thousands of shots per pair of inputs [24]. These two concerns -- energy and cost -- compose multiplicatively with latency to determine whether a workload is viable in production at all.

The framing this thesis adopts is therefore deliberately non-evangelical. Quantum computing is not a panacea. For some workloads -- molecular simulation being the canonical example [28], [29], [30] -- the classical cost grows exponentially in the system size, and even modest fault-tolerant quantum hardware will eventually win on grounds of asymptotic scaling. For other workloads -- low-dimensional tabular classification, of which cybersecurity intrusion detection on classical feature sets is a representative example -- the classical methods are already extremely efficient, and the constant factors of quantum hardware (shots, queue time, coherence-limited circuit depth) dominate. The interesting research question is therefore not the binary 'is quantum better?' but the fine-grained 'for which workload, on which cloud tier, with what trade-off between cost, energy, latency, and accuracy?'

It is this question that motivates the present work. We construct a measurement framework that treats CPU, GPU, and QPU as interchangeable cloud tiers; we instrument it to record per-task accuracy, latency, energy, and 2026 cloud-cost figures; and we apply it to two contrasting workloads -- one in which classical methods are expected to dominate (cybersecurity intrusion detection) and one in which quantum methods will eventually dominate (molecular ground-state estimation). The intent is to replace speculation with measurement and to give cloud architects, quantum researchers, and cybersecurity practitioners a defensible basis on which to make tier-allocation decisions today and to re-measure tomorrow as the hardware evolves.

1.2 Problem Statement

Despite a rapidly growing literature on quantum machine learning, quantum cybersecurity, and energy-aware computing, no published study to our knowledge measures the joint trade-off between accuracy, latency, energy, and cost across heterogeneous cloud tiers (CPU, GPU, QPU) for hybrid quantum-classical workloads. The available evaluations almost invariably fix one of these axes and vary the others. Quantum machine-learning papers typically report accuracy on a small benchmark and stop; cloud benchmarking papers typically report latency or throughput on a single tier; energy studies typically focus on training-time energy of large neural networks. The composite question -- how does a hybrid pipeline behave when one demands a simultaneous accounting of all four metrics across all three tiers -- has not been answered.

The consequence is that cloud architects who must decide how to allocate a hybrid quantum-classical pipeline have no published methodology to draw on. Should the data-loading stage run on CPU or GPU? Should kernel evaluation run on CPU or QPU? Should the variational optimiser be co-located with the QPU back-end or kept on a CPU client? The honest answer today is that these decisions are made by intuition and vendor advocacy rather than by measurement. The lack of an open, reproducible, instrumented framework that one can run end-to-end against a real workload is a concrete technical gap, and it is the gap this thesis addresses.

We can therefore state the central problem in a single sentence:

Given a quantum-AI workload of type T and size n, on which cloud tier (CPU, GPU, or QPU) should it be executed in order to optimise the joint objective of accuracy, latency, energy, and cost, subject to user-supplied weights?

Answering this question requires three things that, taken together, constitute the technical content of this thesis. First, an orchestration framework that can dispatch a workload to any of the three tiers without source-level changes. Second, a measurement harness that records all four metrics per task and aggregates them across statistically meaningful repetitions. Third, an allocation rule that consumes these measurements together with user-supplied weights and returns a recommended tier with a quantitative confidence margin.

1.3 Research Questions

The problem statement decomposes naturally into four research questions, each of which is taken up by a specific chapter of this thesis.

RQ1. How does the accuracy-latency-energy trade-off differ between classical and quantum methods on cybersecurity intrusion detection? The expectation, motivated by the low feature dimensionality and the maturity of classical kernel and ensemble methods, is that classical approaches will dominate. The research task is to quantify by how much, and to characterise the trade-off precisely enough that an architect can defend the choice numerically.

RQ2. How does the same trade-off differ on molecular ground-state estimation, where the classical cost is exponential in problem size? Here the expectation is reversed in the asymptotic limit, but for the H2 molecule used in our measurements the system is small enough to be solved exactly by classical diagonalisation. The research task is to measure the constant-factor overhead of the

variational quantum eigensolver today and to project where the cross-over with classical exact methods will occur.

RQ3. Can a workload-aware allocation rule, calibrated from measured per-task costs, recommend the optimal cloud tier with quantitative confidence? The research task is to formulate such a rule (Algorithm 1 of this thesis), to validate that its recommendations agree with a Pareto-frontier analysis of the raw measurements, and to expose the user-tunable weights so that the recommendation can be steered toward cost-, energy-, latency-, or accuracy-prioritised regimes.

RQ4. How do these trade-offs translate into 2026 cloud-pricing dollar figures across AWS, IBM Quantum, and Microsoft Azure? Energy and latency are scientifically interesting but it is ultimately the dollar cost that determines what gets deployed. The research task is to construct a transparent cost model spanning the three providers, to apply it to our measurements, and to report cost per task and cost per million tasks in a form directly usable by procurement.

1.4 Contributions

This thesis makes seven principal contributions, listed here and elaborated in the chapters indicated.

1. The HERO framework. We introduce the Heterogeneous Energy-aware Runtime Orchestrator, a software framework for CPU+GPU+QPU orchestration of hybrid quantum-classical workloads. HERO dispatches each task to a chosen tier, instruments it for accuracy, latency, energy, and cloud cost, and writes structured measurement records that are aggregated across repetitions. The framework is described in detail in Chapter 7.

2. Two contrasting case studies measured under the same methodology. We apply HERO to cybersecurity intrusion detection on the KDD Cup 99 dataset (Chapter 9) and to molecular ground-state estimation of the H2 molecule via the Variational Quantum Eigensolver (Chapter 10). The use of identical measurement infrastructure across two workloads with opposite expected verdicts gives the cross-workload comparison in Chapter 11 unusual scientific weight.

3. Comparison of five classifiers on cybersecurity. For the intrusion-detection case study we benchmark Random Forest, Gradient Boosting, K-Nearest Neighbors, classical Support Vector Machine, and Quantum Support Vector Machine, with all five evaluated under 30 independent runs to support statistical validation by Welch's t-test and confidence-interval reporting.

4. A 2026 cloud cost model. We construct a transparent cost model spanning AWS EC2 general-purpose instances, AWS g5 and p5 GPU instances, AWS Braket QPU access, IBM Quantum Premium plans, and Microsoft Azure Quantum hardware providers. The model is presented as cost per task and cost per million tasks and is applied uniformly to both workloads.

5. Algorithm 1: workload-aware tier allocation. We formulate a tier-allocation rule that takes a workload descriptor and user-supplied weights over cost, energy, latency, and accuracy and returns a recommended tier together with a confidence margin. The algorithm is validated against the Pareto frontier computed directly from the measurement records.

6. A crossover projection. Combining our measured per-task costs with the known O(d) classical and O(log d) fault-tolerant quantum scaling of kernel evaluation, we project the feature dimension at which quantum kernel methods become cost-competitive for cybersecurity workloads. This projection is presented transparently with its assumptions so that the reader can substitute their own pricing as the market evolves.

7. Open-source release. The HERO simulator, the allocation rule, the Pareto-selection module, the cloud cost calculator, and all measurement scripts used to produce the figures in this thesis are released under a permissive open-source licence so that the community can re-measure as quantum hardware improves.

1.5 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 provides a self-contained introduction to quantum computing, beginning from the definition of a qubit and developing the necessary background on multi-qubit systems, gates, circuits, measurement, and the Noisy Intermediate-Scale Quantum (NISQ) regime in which our experiments are conducted.

Chapter 3 introduces the four quantum algorithms relevant to this thesis -- Grover's search, the Quantum Approximate Optimization Algorithm, the Quantum Support Vector Machine, and the Variational Quantum Eigensolver -- and locates each on the map of HERO workloads.

Chapter 4 reviews the classical machine-learning background needed to understand the cybersecurity baselines: kernel methods and Support Vector Machines, Random Forest and Gradient Boosting, K-Nearest Neighbors, the standard evaluation metrics, and the multi-run statistical validation methodology.

Chapter 5 surveys the cloud computing and cybersecurity context: heterogeneous cloud tiers, the principal cloud quantum services, the network intrusion-detection problem, and energy and Power Usage Effectiveness in datacentres.

Chapter 6 conducts a literature review and gap analysis spanning quantum machine learning, quantum cybersecurity, hybrid quantum-classical orchestration, energy-aware computing, and cloud quantum cost studies, and positions HERO within that landscape.

Chapter 7 presents the HERO framework design in full: architecture, tier roles and dispatch, energy model, cloud cost model, the workload allocation algorithm, and the Pareto selection method.

Chapter 8 describes the implementation: software stack, module structure, reproducibility procedures, and the limitations of the current implementation.

Chapter 9 reports the cybersecurity workload results: dataset preprocessing, the QSVM pipeline, the classical baselines, accuracy-latency-energy results, the cloud cost analysis, and discussion.

Chapter 10 reports the molecular simulation workload results: the H2 Hamiltonian, the hardware-efficient ansatz, the VQE pipeline, the classical exact diagonalisation baseline, results on energy convergence and resource cost, and the projected scaling to larger molecules.

Chapter 11 conducts the cross-workload analysis, presents the Pareto frontier, applies the workload-aware allocation rule, and develops the crossover projections.

Chapter 12 discusses the implications of these findings for cloud architects, quantum researchers, and cybersecurity practitioners, and addresses threats to validity.

Chapter 13 concludes with a summary of contributions, limitations, and a structured agenda for future work covering real hardware re-measurement, additional workloads, and production deployment.

Appendices A-D provide code listings, a reproducibility guide, additional plots, and extended results tables.

Chapter 2 — Quantum Computing From Zero

2.1 The Qubit

A classical bit is a deterministic two-state system, copyable and observable without disturbance. The qubit relaxes all three of these assumptions. Formally, a qubit is a unit-norm vector in a two-dimensional complex Hilbert space with orthonormal basis {|0>, |1>}, and an arbitrary pure state is a complex linear combination of these basis vectors:

$$|psi> = alpha\ |0> + beta\ |1>, \quad with \quad |alpha|^2 + |beta|^2 = 1.$$

The complex amplitudes alpha and beta are not themselves probabilities, but their squared moduli are: the Born rule (Section 2.5) states that a computational-basis measurement returns 0 with probability |alpha|^2 and 1 with probability |beta|^2. The Hadamard basis {|+>, |->} = {(|0>+|1>)/sqrt(2), (|0>-|1>)/sqrt(2)} and the Y basis are alternative orthonormal bases used elsewhere in the thesis; the relative phases between amplitudes are physically real and are the resource that interference exploits.

Up to an undetectable global phase every pure single-qubit state can be written

$$|psi> = cos(theta/2)\ |0> + e^{i\ phi}\ sin(theta/2)\ |1>,$$

with (theta, phi) the polar and azimuthal coordinates of a point on the unit Bloch sphere. Single-qubit unitary operations correspond to rotations of this sphere; decoherence (Section 2.6) shrinks the state vector inward. The Bloch picture is exact only for single qubits but is the standard mental model used throughout this thesis.

Figure 2.1: Bloch sphere representation of a single-qubit pure state |psi> = cos(theta/2) |0> + e^{i phi} sin(theta/2) |1>. (TODO: figure to be inserted in Phase 7.)

2.2 Multi-Qubit Systems and Entanglement

Multi-qubit registers are built by the tensor product. If qubit A is in state |psi_A> and qubit B in state |psi_B>, the joint state of the independent pair is

$$|Psi\_AB> = |psi\_A> \ tensor\ |psi\_B>,$$

with computational basis {|00>, |01>, |10>, |11>}. For an n-qubit register the Hilbert space has complex dimension 2^n, and an arbitrary pure state is

$$|Psi> = sum\_\{x\ in\ \{0,1\}^n\}\ alpha\_x\ |x>, \quad sum\_x\ |alpha\_x|^2 = 1.$$

The exponential 2^n dimensionality is both the promise and the simulation cost of quantum computing: an n-qubit unitary acts on all 2^n amplitudes in parallel, but classical state-vector simulation becomes infeasible around n = 50 and n = 60 even on the largest supercomputers.

Most n-qubit states are not separable -- they cannot be written as a tensor product of single-qubit states. The canonical example is the two-qubit Bell state

$$|Phi+> = (1/sqrt(2))\ (|00> + |11>),$$

one of the four maximally entangled Bell states. Operationally, entanglement is a resource: a measurement of one qubit in |Phi+> instantly fixes the outcome of a subsequent measurement of the other. The CNOT gate applied to a control qubit in superposition and a target in |0> generates exactly this kind of entanglement, which is what gives the variational ansaetze used by VQE in Chapter 3 a richer reachable subspace than any product state could span.

2.3 Quantum Gates

A quantum gate is a unitary operator on the Hilbert space of one or more qubits. Unitarity (U U^dagger = I) preserves norm and makes gate-only quantum computation reversible, in contrast to most classical logic gates.

The single-qubit Pauli operators are

$$I = [[1, 0], [0, 1]], \quad X = [[0, 1], [1, 0]],$$

$$Y = [[0, -i], [i, 0]], \quad Z = [[1, 0], [0, -1]].$$

X is the quantum NOT (|0> <-> |1>); Z is the phase-flip (sign-flip on |1>); Y = i X Z. Each is a 180-degree rotation of the Bloch sphere about its eponymous axis.

The Hadamard gate creates superposition:

$$H = (1/sqrt(2)) [[1, 1], [1, -1]],$$

with H|0> = |+>, H|1> = |->. Applied to every qubit of |00...0> it produces the uniform superposition over all 2^n computational-basis states, the standard first step of nearly every quantum algorithm. The diagonal phase gates S and T add non-real phases to |1> and complete the standard gate library.

Continuous rotations of the Bloch sphere are generated by the Pauli operators:

$$RX(theta) = exp(-i theta X / 2), \quad RY(theta) = exp(-i theta Y / 2), \quad RZ(theta) = exp(-i theta Z / 2).$$

Every trainable parameter in the hardware-efficient VQE ansatz of Chapter 10 is the angle of one of these single-axis rotations.

Two-qubit entanglement is introduced by the controlled-NOT (CNOT), which flips the target iff the control is |1>:

$$CNOT = [[1,0,0,0], [0,1,0,0], [0,0,0,1], [0,0,1,0]]$$

(basis ordering |00>, |01>, |10>, |11>). Controlled-Z and SWAP decompose into CNOTs and single-qubit gates. The set {H, T, CNOT} is universal: by the Solovay-Kitaev theorem any single-qubit unitary can be approximated to precision epsilon by an H/T sequence of length polylog(1/epsilon), and CNOT promotes the set to universality on multi-qubit systems.

Variational algorithms instead fix a parameterised circuit structure -- alternating layers of single-qubit rotations and CNOT entanglers -- and treat the rotation angles as continuous trainable parameters. These parameterised quantum circuits are the building blocks of QAOA, VQE, and QSVM in Chapter 3.

2.4 Quantum Circuits

A quantum circuit is a graphical notation for a sequence of gates applied to a register of qubits: one horizontal wire per qubit, time flowing left to right, gates drawn as labelled boxes intersecting the wires they act on. The left end of the diagram is the initial state (almost always |00...0>) and the right end is the final state or, where measurement symbols appear, the classical bits produced.

The canonical small example is Bell-state preparation:

```
|0> --[ H ]--*------     (qubit 0)
|0> ---------X------     (qubit 1)
```

A Hadamard on qubit 0 puts the register in (|00> + |10>)/sqrt(2); the CNOT then maps |10> to |11>, producing the Bell state |Phi+> = (|00> + |11>)/sqrt(2).

Figure 2.2: Quantum circuit notation example. The two-qubit Bell-state preparation circuit applies a Hadamard to qubit 0 followed by a CNOT with qubit 0 as control and qubit 1 as target, producing the entangled state (1/sqrt(2))(|00> + |11>). (TODO: figure to be inserted in Phase 7.)

Every quantum algorithm has three logical phases: state preparation (initialise the register from |00...0>), evolution (apply the algorithm-specific unitary U, often as many layers of parameterised gates), and measurement (read out classical bits, which may be used directly or fed back to a classical co-processor as in QAOA and VQE).

2.5 Measurement

Measurement is the bridge between unitary evolution and classical bits. It is irreversible, probabilistic, and destroys information about the un-measured components of the state. The Born rule fixes the outcome probabilities: given a state |psi> and a measurement basis, the outcome associated with basis vector |x> appears with probability

$$P(x) = |\langle x \mid psi \rangle|^2.$$

For a single qubit in the computational basis this gives P(0) = |alpha|^2 and P(1) = |beta|^2; immediately after measurement the qubit collapses to the corresponding eigenvector and the original superposition is gone.

Because each measurement returns a single sample, estimating any expectation value requires preparing the state and measuring repeatedly. Each repetition is a shot, and the standard error of a Bernoulli estimate from N shots scales as 1/sqrt(N). On real hardware each shot is billed individually, so choosing the shot count is a precision-cost trade-off. Measurement need not be in the computational basis: applying a basis-change unitary (e.g. H for the X basis, S^dagger then H for the Y basis) before a computational-basis measurement lets a fixed-basis device read out arbitrary bases.

These considerations dictate the shot counts used in HERO. For QSVM kernel evaluation (Chapter 9) we use 2000 shots per kernel entry, giving a standard error of about 1/sqrt(2000) ~ 0.022, small relative to the dynamic range of the kernel and not ruinously expensive in cloud-cost terms. For VQE energy estimation (Chapter 10) we use 4000 shots per Pauli term to keep the aggregate variance below the chemical accuracy threshold of 1.6 milli-Hartree.

2.6 Decoherence and the NISQ Era

Real qubits are coupled to their environment, and this coupling causes their quantum information to leak away. The leakage is conventionally split into two timescales: the relaxation time T1 (energy loss, |1> -> |0>) and the dephasing time T2 (loss of phase coherence between |0> and |1> components), with the constraint 1/T2 <= 2/T1.

Figure 2.3: Decoherence timescales. A qubit prepared in |1> relaxes to |0> with time constant T1, and a qubit prepared in a superposition loses phase coherence with time constant T2 (illustrative). (TODO: figure to be inserted in Phase 7.)

Imperfect gates compound multiplicatively along a circuit, limiting circuit depth before noise dominates the signal. John Preskill named the resulting regime Noisy Intermediate-Scale Quantum, or NISQ [2]: hardware with roughly 50 to 1000 qubits, no full quantum error correction, and depths bounded by accumulating errors. NISQ-era algorithms (VQE, QAOA, QSVM) keep circuits shallow and rely on classical post-processing or variational self-correction.

Throughout this thesis we use the noiseless Aer simulator from Qiskit [25] for reproducible, hardware-independent results. The consequence, which we return to in Chapter 12, is that our reported quantum accuracy figures are best-case; noisy-hardware re-measurement is a primary future-work item in Chapter 13.

2.7 Why This Matters for the Thesis

Recapping: a qubit is a unit-norm vector in C^2 (Bloch sphere); an n-qubit register lives in the 2^n-dimensional tensor-product space and can carry entanglement; computation is unitary; measurement is Born-rule probabilistic; current hardware sits firmly in the NISQ regime.

Each of these ideas connects forward. The parameterised rotations RX, RY, RZ (Section 2.3) and the multi-qubit Hilbert-space structure (Section 2.2) are the building blocks of the four algorithms in Chapter 3 and of the cybersecurity QSVM and molecular VQE case studies in Chapters 9 and 10. The Born-rule shot analysis (Section 2.5) directly justifies the 2000- and 4000-shot budgets used in those chapters. The decoherence picture (Section 2.6) sets the interpretation limits of our noiseless-simulator results.

We turn in the next chapter to the four quantum algorithms the HERO framework actually executes -- Grover's search, QAOA, QSVM, and VQE -- and explain why QSVM and VQE were chosen as the case studies driving the rest of the thesis.

Chapter 3 — Quantum Algorithms Explained

3.1 Grover's Search

Grover's algorithm (1996) finds a marked element in an unstructured space of size N = 2^n quadratically faster than any classical algorithm. Given a Boolean oracle f: {0,1}^n -> {0,1} with a unique x* such that f(x*) = 1, classical search requires O(N) queries; Grover finds x* with high probability in O(sqrt(N)) oracle queries, which is provably optimal for unstructured search [9].

Algorithm Overview

- Initialise an n-qubit register in the equal superposition |s> = H^{tensor n} |0>^{tensor n} = (1/sqrt(N)) sum_x |x>.
- Apply k = floor((pi/4) sqrt(N)) iterations of the Grover operator G = D U_f.
- Measure in the computational basis; the outcome is x* with probability close to one.

The oracle marks the target by a phase flip,

$$U_f |x\rangle = (-1)^{f(x)} |x\rangle,$$

and the diffusion operator is the reflection about the uniform superposition,

$$D = 2 |s\rangle\langle s| - I.$$

G = D U_f is a product of two reflections and therefore a rotation in the plane spanned by |x*> and the uniform superposition over non-targets. Each iteration rotates the state vector by 2 theta with sin(theta) = 1/sqrt(N); after k iterations the success probability is sin^2((2k+1) theta), maximised at k_opt = floor((pi/4) sqrt(N)). Continuing past k_opt rotates the state away from |x*>, so more iterations are not better.

Figure 3.1: Grover's amplitude amplification geometry. Each Grover iteration rotates the state vector by 2 theta in the plane spanned by the target |x> and the uniform superposition of non-targets, where sin(theta) = 1/sqrt(N). (TODO: figure to be inserted in Phase 7.)*

The quadratic speedup assumes oracle access: in any concrete application the cost of constructing U_f as a quantum circuit is part of the budget and may dominate. On NISQ hardware the depth needed for k iterations of D may also exceed coherence budgets for moderate n, so empirical advantage is not guaranteed.

We use Grover in Chapter 9 as a cryptographic-search baseline; HERO measures its per-task energy and runtime alongside the classical brute-force comparison.

3.2 Quantum Approximate Optimization Algorithm (QAOA)

QAOA, introduced by Farhi, Goldstone and Gutmann in 2014 [5], is a hybrid quantum-classical algorithm for combinatorial optimisation on near-term hardware. It targets the ground state of a problem-specific cost Hamiltonian whose minimum encodes the optimal configuration, while outsourcing the parameter search to a classical outer-loop optimiser.

MaxCut Encoding

MaxCut -- partition the vertices of a graph G = (V, E) into two sets so as to maximise the number of cut edges -- is the canonical QAOA testbed. It is NP-hard in general; the Goemans-Williamson SDP relaxation achieves an approximation ratio of about 0.878 in polynomial time. Assigning one qubit per vertex and letting z_i in {0, 1} encode the side, the cost Hamiltonian is

$$H_C = \sum_{(i,j) \in E} (1/2) (I - Z_i Z_j),$$

with mixer

$$H_M = \sum_i X_i,$$

whose ground state is the equal superposition $|+\rangle^{\otimes n}$.

The QAOA Ansatz

The depth-p QAOA ansatz interleaves p cost evolutions with p mixer evolutions:

$$|\psi(\beta, \gamma)\rangle = U_M(\beta_p)\, U_C(\gamma_p) \cdots U_M(\beta_1)\, U_C(\gamma_1)\, |s\rangle,$$

with $U_C(\gamma) = \exp(-i\gamma H_C)$, $U_M(\beta) = \exp(-i\beta H_M)$, $|s\rangle = |+\rangle^{\otimes n}$, and 2p real parameters. Larger p is more expressive but deeper.

Figure 3.2: QAOA depth-p ansatz circuit diagram. After Hadamard initialisation the cost-evolution block U_C(gamma_k) and mixer block U_M(beta_k) are applied in alternation for p layers. (TODO: figure to be inserted in Phase 7.)

Hybrid Outer Loop and Approximation Ratio

A classical optimiser tunes (β, γ) to minimise $\langle\psi(\beta,\gamma)| -H_C |\psi(\beta,\gamma)\rangle$; each evaluation requires preparing the ansatz on the QPU and averaging per-shot cut values. COBYLA (Powell's gradient-free method) is the standard choice because it handles the noisy black-box objective in few function evaluations; Nelder-Mead and SPSA are also used. The approximation ratio $\alpha = E[\text{cut}(x_{QAOA})] / \text{cut}_{opt}$ satisfies $\alpha \geq 0.6924$ for $p = 1$ on 3-regular graphs [5]; at finite p the achievable ratio depends on graph structure, but $p = 2$ or $p = 3$ already gives competitive ratios on small graphs at depths shallow enough for noisy hardware.

We use QAOA for MaxCut on synthetic network-segmentation graphs in Chapter 9; HERO records 30-seed approximation-ratio statistics together with per-task latency and energy.

3.3 Quantum Support Vector Machine (QSVM)

The Quantum Support Vector Machine considered in this thesis is a hybrid algorithm that uses a quantum circuit to compute a kernel function and hands the resulting kernel matrix to an entirely classical SVM solver. The quantum contribution is isolated to a single object -- the kernel -- which makes the comparison against classical kernels exceptionally clean.

Classical SVM and the Kernel Trick

A binary SVM finds the maximum-margin hyperplane separating two classes; its dual problem depends on training data only through inner products $\langle x_i, x_j\rangle$. The kernel trick replaces this inner product by any positive-semidefinite function $K(x, x') = \langle\phi(x), \phi(x')\rangle$, implicitly mapping the data into a (possibly infinite-dimensional) feature space.

Quantum Feature Maps and the ZZ Map

A quantum feature map is a parameterised circuit $\phi(x)$ that encodes a classical input $x \in \mathbb{R}^d$ into an n-qubit state $|\phi(x)\rangle$. The induced kernel is the squared overlap

$$K(x, x') = |\langle\phi(x) | \phi(x')\rangle|^2.$$

The most widely used near-term feature map is the second-order Pauli-Z (ZZ) map of Havlicek et al. 2019 [6]. With one qubit per feature, each layer applies a Hadamard on every qubit, a single-qubit phase $RZ(2 x_i)$ per feature, and for every pair (i, j) an entangling block $CX(i, j); RZ(2 (\pi - x_i)(\pi - x_j))$ on j; $CX(i, j)$. The non-linear $(\pi - x_i)(\pi - x_j)$ factor is the source of the feature interaction that is argued to be classically inefficient to simulate.

Figure 3.3: ZZ feature-map circuit on 4 qubits, depth 2. Hadamards initialise the equal superposition; single-qubit RZ(2 x_i) phases encode each feature; CX-RZ-CX entangling blocks introduce the second-order interaction (pi - x_i)(pi - x_j). (TODO: figure to be inserted in Phase 7.)

Kernel Cost and Hybrid Pipeline

Each kernel entry is estimated by running the inversion circuit phi(x')^dagger phi(x) on |0>^n and measuring the all-zero probability with S shots (standard error 1/sqrt(S)). An N-point symmetric kernel matrix has N(N+1)/2 distinct entries, so total quantum cost scales as O(N^2) -- the practical bottleneck that drives our quantum sample-size limit in Chapter 9. The classical SVM solver (libsvm via scikit-learn) consumes K to produce the dual coefficients alpha_i and bias b; inference on a new point x evaluates K(x, x_i) for each support vector.

Theoretical Advantage

Liu, Arunachalam and Temme [36] proved in 2021 that for a constructed family of data distributions related to the discrete-logarithm problem, no classical algorithm can match the quantum-kernel classifier in polynomial time under standard cryptographic assumptions. The result is a genuine separation but on a constructed dataset; on natural data the question of advantage remains empirical.

QSVM appears in two HERO chapters: as a candidate route in the Chapter 9 cybersecurity scheduler (HERO-S workflow) and as the intrusion-detection classifier whose energy and latency are measured.

3.4 Variational Quantum Eigensolver (VQE)

The Variational Quantum Eigensolver (Peruzzo et al. 2014) is a hybrid algorithm for estimating the ground-state energy of a Hamiltonian. Its primary application is molecular electronic structure: the Hamiltonian is the second-quantised molecular Hamiltonian after a fermion-to-qubit mapping, and the ground-state energy underpins prediction of geometries, reaction energetics, and binding constants in chemistry and drug discovery. Classical exact diagonalisation scales as 2^N in the number of spin-orbitals; VQE side-steps this wall by representing the state on N qubits.

Variational Principle

For any normalised state |psi> the expectation of H is bounded below by the ground-state energy E_0:

$$\langle psi \mid H \mid psi \rangle \geq E_0,$$

with equality iff |psi> is a ground state. Parameterising |psi(theta)> and minimising over theta yields an upper bound on E_0 that tightens as the ansatz becomes expressive enough.

Algorithm

- Prepare |psi(theta)> = U(theta) |0>^n on the QPU.
- Decompose H = sum_k c_k P_k into weighted Pauli strings.
- Estimate <H> = sum_k c_k <P_k> by sampling each term.
- Pass <H>(theta) to a classical outer optimiser (COBYLA in this thesis) to propose a new theta; iterate to convergence.

The ansatz controls expressivity and trainability. We use the hardware-efficient ansatz of Kandala et al. [29], which alternates layers of single-qubit RY (and optionally RZ) rotations with native two-qubit entanglers, scales linearly in qubits and layers, and is shallow enough to fit current coherence budgets.

H2 Benchmark

The hydrogen molecule H2 at 0.735 Angstrom in the minimal STO-3G basis maps (after Bravyi-Kitaev with two-qubit reduction) to a five-term two-qubit Hamiltonian with exact ground-state energy E_0 = -1.857275 Hartree. A four-parameter two-qubit hardware-efficient ansatz reaches

|E_VQE - E_0| < 0.001 Hartree, well within chemical accuracy (1.6 mHa), in 80-150 COBYLA iterations with 4000 shots per Pauli term.

Figure 3.4: VQE energy convergence for H2 (illustrative). The estimated energy decreases from a random initial value and asymptotes to within chemical accuracy of the exact ground-state energy after roughly 100 COBYLA iterations. (TODO: figure to be inserted in Phase 7.)

We use VQE on H2 in Chapter 10 as the contrast workload to cybersecurity QSVM, illustrating a regime where quantum methods are essential at scale even though classical methods win on the H2 test case itself.

3.5 Algorithm Map for HERO Workloads

The four algorithms presented in this chapter are the complete repertoire of quantum methods that the HERO framework executes. The table below summarises where each appears in the empirical chapters of the thesis.

- Grover's Search -- Chapter 9, cryptographic-search baseline. We measure per-task energy, latency, and shot count for fixed-N unstructured search instances against a classical brute-force comparator.
- QAOA -- Chapter 9, MaxCut on synthetic network-segmentation graphs. Depth p in {1, 2, 3} is swept; classical greedy and Goemans-Williamson serve as baselines.
- QSVM with ZZ feature map -- Chapter 9, intrusion-detection classifier on four IDS datasets, and as the QPU-capable route in the HERO-S scheduler.
- VQE -- Chapter 10, ground-state energy estimation for the H2 molecule using a hardware-efficient ansatz, contrasted with full configuration interaction as the classical reference.

It is worth emphasising what the algorithm map is not. HERO is not tied to these four algorithms in any architectural sense. The scheduler accepts as input an abstract route contract -- predicted classification quality, confidence, estimated latency, estimated energy, and current queue state for each candidate route -- and chooses among the routes by the criteria laid out in Chapter 8. Any quantum (or classical) algorithm that can populate the route contract for a given task can plug into HERO without modification of the scheduler itself. The four algorithms above were chosen because they are mature, widely benchmarked, and span the two HERO case-study workloads; the framework is designed to outlive them.

Chapter 4 — Classical AI/ML Background

4.1 Support Vector Machines and Kernel Methods

The classical SVM (Cortes and Vapnik 1995) is a maximum-margin classifier and the natural classical comparator for the quantum SVM of Section 3.3 -- the two differ only in how the kernel is computed. The kernel trick replaces the explicit inner product <x_i, x_j> by any positive-semidefinite K(x_i, x_j) = <phi(x_i), phi(x_j)>, implicitly mapping data into a (possibly infinite-dimensional) feature space. The dominant classical kernel is the Radial Basis Function,

$$K(x, y) = \exp(-\gamma \|x - y\|^2),$$

which maps into an infinite-dimensional Gaussian feature space. Training cost is O(N^2 d) to fill the kernel matrix plus O(N^2) to O(N^3) to solve, which is efficient on a single core for N up to a few tens of thousands. We use linear and RBF SVMs as primary classical comparators throughout the HERO experiments.

4.2 Random Forest and Gradient Boosting

Tree-ensemble methods are the de facto strongest classical baseline on tabular intrusion-detection data, consistently outperforming SVMs and shallow neural networks on benchmarks such as NSL-KDD, CICIDS-2017, and UNSW-NB15. Two families dominate.

Random Forest

Breiman's Random Forest (2001) [15] is an ensemble of T decision trees that vote on the prediction. Each tree is trained on a bootstrap sample with a random subset of features considered at each split; this double randomisation decorrelates the trees, reduces variance, and is embarrassingly parallel to train. Random forests are insensitive to feature scaling, tolerate missing values, and expose feature-importance scores -- operationally valuable for security analysts.

Gradient Boosting

Gradient Boosting Machines (Friedman 2001) instead fit shallow trees sequentially, each new tree targeting the negative gradient of the loss with respect to the current ensemble's predictions. The result is a stagewise additive model whose dominant hyperparameters are the number of estimators, tree depth, and learning rate. On tabular cybersecurity benchmarks gradient-boosted trees are the methods to beat; HERO uses them as the primary classical baseline alongside random forest and the SVM family.

4.3 K-Nearest Neighbors

K-Nearest Neighbors (k-NN) predicts the class of a query x_q by majority vote among the k training points closest to x_q under a chosen distance metric (typically Euclidean). It has no training stage to speak of -- the training set is simply stored -- and all work is deferred to inference, where a naive scan costs O(N d) per query. The hyperparameter k controls the bias-variance trade-off and is selected by cross-validation. k-NN is highly sensitive to feature scaling, so standardisation to zero mean and unit variance is mandatory preprocessing. We include k-NN in HERO as a memory-bound, latency-heavy classical baseline whose predictable energy profile is useful in the comparisons of Chapter 9.

4.4 Evaluation Metrics: Accuracy, Precision, Recall, F1

The binary confusion matrix records four counts -- true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN) -- from which the headline metrics derive: Accuracy = (TP+TN)/(TP+FP+TN+FN), Precision = TP/(TP+FP), Recall = TP/(TP+FN), and F1 = 2 P R / (P + R). Accuracy is misleading on imbalanced IDS data because a trivial all-benign predictor scores the prevalence of the benign class. F1 is our headline metric, but precision and recall are always reported alongside because the operational cost of false negatives (missed attacks) is qualitatively different from that of false positives (alert fatigue).

4.5 Multi-Run Statistical Validation

Single-run benchmark numbers mislead: on typical IDS data the train/test split seed alone shifts F1 by two to four percentage points, larger than many published differences. HERO therefore reports every (workload x method) cell as a distribution over 30 independent seeds, shared across methods within a workload so the comparisons are paired. For each cell we report mean +/- std and the empirical distribution as a box plot in Chapters 9 and 10.

Pairwise significance between two methods is tested with the Wilcoxon signed-rank test on the 30 paired F1 values. We declare significance at $p < 0.05$ with Bonferroni correction across the four IDS datasets (per-dataset threshold $p < 0.0125$); the full per-comparison p-value table appears in Appendix B.

Chapter 5 — Cloud Computing and Cybersecurity Context

5.1 Heterogeneous Cloud Tiers (CPU, GPU, QPU)

Cloud computing in the Infrastructure-as-a-Service and Platform-as-a-Service sense supplies on-demand virtualised compute, storage, and accelerator resources that the customer pays for by the second, the minute, or the hour. The HERO orchestrator treats the cloud not as a single homogeneous pool of compute but as three contrasting tiers, each with its own price, energy footprint, latency profile, and class of workloads on which it excels. The first contribution of this chapter is to fix the vocabulary for those three tiers so that the measurement chapters can refer to them without ambiguity.

CPU Tier

The CPU tier consists of general-purpose virtual machines exposed by every major hyperscaler -- AWS EC2, Microsoft Azure (D-series and F-series), and Google Cloud Platform (n2-standard and c3-standard families). Representative for HERO's experiments is an AWS EC2 c7i.4xlarge instance with 16 vCPUs (Intel Xeon Sapphire Rapids), 32 GB of DDR5 memory, and a list price of $0.7140 per hour as of April 2026. Such instances are the historical workhorse of the cloud: they handle web serving, database queries, batch ETL, and small-to-medium machine-learning jobs at predictable cost.

GPU Tier

The GPU tier consists of accelerator-equipped instances aimed at deep learning, scientific simulation, and large-batch linear algebra. Examples include AWS g5 and p5, Azure ND H100 v5, and GCP A3. At the entry level the AWS g5.xlarge ships a single NVIDIA A10G GPU and bills $1.006 per hour; at the high end the AWS p5.48xlarge ships eight NVIDIA H100 GPUs over NVLink and bills $98.32 per hour. The order-of-magnitude difference in price within the same tier matters: the allocation rule of Chapter 7 must select a representative GPU price point and justify the choice against the workload's memory footprint.

QPU Tier

The QPU tier is the youngest and the most heterogeneous. IBM Quantum exposes superconducting transmon backends through Qiskit Runtime; the Eagle device offers 127 qubits at $1.60 per second on the Premium plan. AWS Braket aggregates several vendors, including the Rigetti Ankaa superconducting device (84 qubits, $0.30 per task plus $0.0009 per shot) and the IonQ Aria trapped-ion device (25 qubits, $0.30 per task plus $0.03 per shot). Microsoft Azure Quantum exposes the Quantinuum H2 trapped-ion device with 56 qubits at $12.50 per minute. Pricing models differ by vendor -- per-second, per-shot, per-minute -- and a uniform comparison is non-trivial; Section 7.4 normalises these into a single per-task figure.

Figure 5.1: HERO cloud-tier hierarchy. CPU tier (general-purpose VMs), GPU tier (accelerator instances), and QPU tier (quantum-as-a-service backends). Each tier has its own billing model and representative power draw.

The tier-selection problem is the central operational question this thesis addresses. Every workload now carries an allocation choice, and the existing literature does not provide a unified cost-energy-latency-accuracy decision rule that spans all three tiers. The remainder of this chapter provides the background -- cloud quantum services in Section 5.2, the cybersecurity application domain in Section 5.3, and datacenter energy in Section 5.4 -- that the rest of the thesis assumes.

5.2 Cloud Quantum Services: IBM Quantum, AWS Braket, Azure Quantum

Cloud-accessible quantum computing is less than a decade old. Until 2016 quantum hardware lived almost exclusively in university and national-laboratory cryostats and was reachable only through bilateral collaborations. The launch of the IBM Quantum Experience in May 2016 -- a five-qubit superconducting device with a free public web interface -- changed the model decisively, and within four years every major cloud provider had a quantum offering of its own.

IBM Quantum

IBM Quantum provides the longest-running and the largest fleet of cloud quantum hardware. Its devices are superconducting transmons fabricated on silicon chips and operated at millikelvin temperatures inside dilution refrigerators. Generations relevant to this thesis include Eagle (127 qubits), Heron (133 qubits with improved gate fidelity), and the in-development Condor (1,121 qubits announced as the next-generation target). Programmes are submitted through Qiskit Runtime [25], which packages variational and primitive-based workloads (Sampler, Estimator) and applies built-in error mitigation. Billing on the Premium plan is per second of quantum runtime.

AWS Braket

AWS Braket is a multi-vendor marketplace rather than a single-hardware platform. It exposes the Rigetti Ankaa superconducting device, the IonQ Aria and Forte trapped-ion devices, and the QuEra Aquila neutral-atom analog device through one Python SDK and one billing relationship. The billing model is per-task plus per-shot: the user pays a fixed task fee for each circuit submitted and an additional per-shot fee scaled by the number of measurement repetitions. Braket also provides Hybrid Jobs, a managed environment for variational algorithms that keeps the classical optimiser co-located with the QPU to reduce round-trip latency.

Microsoft Azure Quantum

Azure Quantum exposes the Quantinuum H-series trapped-ion devices and IonQ hardware through a unified portal. Quantinuum H2 has 56 qubits, all-to-all connectivity, and high gate fidelities; pricing is per minute of QPU time at $12.50 per minute on standard plans. The platform also integrates resource-estimation tooling for fault-tolerant algorithms, although the thesis itself confines its measurements to current near-term hardware.

Practical Concerns Specific to the Cloud QPU

Three cloud-quantum-specific concerns shape the HERO orchestrator. First, backend queue time is rarely negligible: during peak hours a Premium-plan submission to a popular IBM Eagle backend can wait tens of seconds to several minutes before execution begins. Second, every account is subject to shot-budget caps and concurrent-job limits, so a workload that would benefit from a million shots may be forced to split across submissions. Third, there is no low-latency local QPU: the device is invariably on the other side of a wide-area network, so any QPU-capable workload incurs network round-trip plus queue plus actual execution time.

The practical implication for HERO is that any allocation rule that ignores queue time will systematically over-route to the QPU tier. Chapter 7 therefore promotes queue time to a first-class scheduling variable that enters the latency budget alongside execution time and network round-trip.

5.3 Network Intrusion Detection

Network Intrusion Detection Systems (NIDS) sit at the perimeter or in-line within an enterprise network and inspect traffic flows for signatures of malicious behaviour. They are the operational application that motivates the cybersecurity half of HERO's measurement programme: cybersecurity is the largest deployed application of tabular machine learning, the false-negative cost is high, and the latency budget is tight enough that adding a wide-area QPU round-trip is not obviously viable. Quantifying that trade-off is the point of Chapter 9.

Attack Categories

The standard public datasets group attacks into a small number of operational categories. Denial-of-service (DoS) attacks such as SYN flood and UDP flood saturate the target's resources and are typically the most numerous class. Probe or reconnaissance traffic (port scans and similar) precedes most intrusions and is comparatively easy to detect. Remote-to-local (R2L) attacks, including SSH brute force, attempt to obtain user-level access from outside. User-to-root (U2R) attacks escalate from a user-level foothold to administrator privilege. Modern datasets add botnet command-and-

control traffic, web-application attacks, and lateral-movement infiltration that older datasets did not capture.

Datasets

Five datasets dominate the NIDS literature. KDD Cup 99, released in 1999, is historical baseline data and remains in use despite well-documented duplication and class-balance problems. NSL-KDD is a refined and de-duplicated subset of KDD Cup 99 that fixes the most egregious of those issues. CICIDS2017 from the Canadian Institute for Cybersecurity captures realistic modern traffic with web attacks, infiltration, and botnet behaviour. TON_IoT and IoT-23 extend the picture into the Internet-of-Things domain with device-level telemetry and IoT-specific attack patterns.

Evaluation Metrics

Accuracy alone is misleading on intrusion-detection data because the class balance is uneven -- benign traffic dwarfs attack traffic on most realistic captures -- and the false-negative cost is asymmetric: a missed attack causes real damage, while a false alarm merely consumes analyst time. Reporting standards therefore include precision, recall, F1, false-positive rate (FPR), false-negative rate (FNR), and the area under both the ROC and precision-recall curves (ROC-AUC and PR-AUC). HERO reports F1 as its headline metric and supplies the full confusion matrix for every (workload, method) cell in Appendix D.

Figure 5.3: Confusion matrix and the metrics derived from it. True positives, false positives, true negatives, and false negatives feed into precision, recall, F1, FPR, and FNR. On intrusion-detection data the false-negative rate carries the highest operational cost.

Operational Deployment

In production a NIDS sensor sits at a network gateway, captures packets or flow records, normalises them into feature vectors with a tool such as Zeek (formerly Bro) or an equivalent flow exporter, and forwards the vectors to a backend classifier. The latency budget at the gateway is typically 1 to 10 ms per flow if real-time blocking is required, with end-to-end budgets in the hundreds of milliseconds for analyst-loop alerting.

The implication for HERO is direct. Gateway latency and false-negative cost together determine whether a quantum-capable route is worth its added delay. A QPU round-trip that adds 100 ms is unacceptable for in-line blocking but may be acceptable for offline batch reclassification of uncertain flows. Chapter 9 measures this trade-off directly.

5.4 Energy and Power Usage Effectiveness in Datacenters

Datacenter energy accounting is the third pillar of HERO's comparison framework. Cost in dollars and latency in milliseconds are the operational metrics most readily appreciated by industry; energy in joules is the metric that increasingly drives sustainability reporting and procurement decisions. This section defines the quantities that the remainder of the thesis uses and fixes the representative values that the energy model in Section 7.3 plugs into.

Server Power

Modern server CPUs have thermal design powers (TDP) in the 105 to 300 W range per socket, with dual-socket servers reaching 600 W under sustained load. GPU TDP is markedly higher: an NVIDIA A100 dissipates around 300 W and an H100 in SXM form factor up to 700 W. A GPU-accelerated training node therefore consumes several times the energy per unit wall-clock time of an equivalent CPU-only node, although its throughput per joule on suitable workloads is still favourable.

QPU Power

QPU power is dominated not by computation but by the infrastructure that keeps the qubits cold. A single superconducting QPU sits inside a dilution refrigerator that draws on the order of 20 kW continuously, almost independently of whether a circuit is running. Trapped-ion and neutral-atom

systems have lower cooling overhead but additional laser and vacuum infrastructure that ends up in a comparable order of magnitude. The implication is that QPU energy per task is high when the device is under-utilised and falls only when the device is kept saturated -- a fact that the cost model of Chapter 7 makes explicit.

Power Usage Effectiveness

Power Usage Effectiveness (PUE) is the standard metric for datacenter facility efficiency:

$$PUE = \text{total facility power} / \text{IT equipment power}.$$

A PUE of 1.0 would mean that every watt drawn from the grid reaches the IT equipment with no overhead from cooling, lighting, or power conditioning -- a thermodynamic impossibility in practice. Hyperscale operators (Google, AWS, Microsoft Azure) report fleet-average PUE values in the 1.10 to 1.20 range. Older enterprise datacenters typically sit between 1.5 and 2.0. This thesis adopts PUE = 1.2 as a representative hyperscale value throughout, on the grounds that the cloud workloads HERO models run on hyperscale infrastructure.

Figure 5.2: Datacenter energy flow and the PUE multiplier. IT equipment power feeds compute and storage; cooling, power conditioning, and ancillary loads inflate the facility draw by the PUE factor.

Per-Task Energy Estimation

HERO estimates per-task energy through the following first-order model:

$$E_{task} = t_{task} * P_{tier} * PUE \qquad \text{(Equation 1)}$$

where t_task is the wall-clock time of the task in seconds, P_tier is the representative steady-state power draw of the tier in watts, and PUE is the dimensionless facility multiplier. The result E_task is in joules. Section 7.4 multiplies this by the cloud price per joule equivalent to convert energy into dollars.

Live Telemetry Alternatives

Where the platform exposes them, live energy counters give a more accurate per-task figure than Equation 1. Intel CPUs expose the Running Average Power Limit (RAPL) interface, which reports per-package energy counters at sub-millijoule resolution. NVIDIA GPUs expose per-device power and cumulative energy through the NVIDIA Management Library (NVML). Real-QPU facility power is rarely exposed on a per-job basis, so HERO uses modelled values for the QPU tier and live counters where available for the CPU and GPU tiers.

In summary, HERO uses Equation 1 with tier-specific representative powers (CPU 150 W, GPU 300 W, QPU 20 kW) and PUE = 1.2 throughout. Per-tier energy values are then combined with published cloud pricing to convert joules into dollars on a like-for-like basis. The full cost model is presented in Section 7.4.

Chapter 6 — Literature Review and Gap Analysis

6.1 Quantum Machine Learning

Quantum machine learning (QML) splits into two strands relevant to HERO: kernel-based methods that map data into a quantum feature space and hand the kernel to a classical SVM, and variational methods that train parameterised circuits by classical optimisation. The QSVM workload of this thesis belongs to the first strand; VQE in Chapter 10 belongs to the second.

Havlicek et al. 2019 [6] introduced the canonical quantum-kernel construction with the ZZ feature map; Schuld and Killoran 2019 [7] supplied the matching reproducing-kernel Hilbert space framework. Cerezo et al. 2021 [35] is the standard survey of variational quantum algorithms, covering the VQC alternative to kernel methods alongside QAOA and VQE. HERO uses the kernel formulation because it isolates the quantum contribution to the kernel matrix and leaves the rest of the SVM machinery identical to the classical comparator.

6.2 Quantum Cybersecurity

Quantum cybersecurity covers two largely disjoint strands: the cryptanalytic threat that a fault-tolerant quantum computer poses to existing primitives (Shor for RSA/ECC, Grover for symmetric-key security), motivating the NIST post-quantum standardisation effort; and application-level QML for detecting or classifying malicious activity. HERO contributes to the second strand.

QML for security is a small literature. Kalinin and Krundyshev [47] surveyed quantum machine learning techniques for intrusion detection in 2023 and reported encouraging accuracy on small-scale, simulation-only experiments; earlier work applied quantum kernels to malware classification as case studies. None of this prior work measures the cost-energy-latency profile of the quantum option against strong classical baselines on multiple datasets with statistical validation -- the gap Chapter 9 closes.

6.3 Hybrid Quantum-Classical Orchestration

The closest prior work to HERO's orchestration layer is the family of hybrid quantum-classical programming frameworks. These frameworks supply the runtime that lets a classical optimiser drive a quantum subroutine; they do not decide whether the quantum subroutine should be invoked at all. The distinction isolates HERO's positioning: it is a measurement and decision layer above existing orchestrators, not a replacement for them.

On the quantum side, Qiskit Runtime [25] is the most widely deployed framework, exposing Sampler/Estimator primitives with built-in error mitigation and co-locating the classical optimiser with the QPU. AWS Braket Hybrid Jobs offers a similar managed-execution model across multiple vendors, and Pennylane Lightning provides a GPU-accelerated simulator popular for development before hardware submission. On the classical side, Apache Airflow, Prefect, Argo, and Kubernetes/Kubeflow are the dominant DAG and ML schedulers; none is QPU-aware -- they have no concept of a quantum backend, queue model, or cost-energy-latency-accuracy decision rule.

Figure 6.1: HERO's position in the orchestration stack. Existing tools (Qiskit Runtime, Braket Hybrid Jobs, Airflow, Kubernetes) handle programming and execution. HERO sits above them as a measurement and decision layer that decides which tier each task should run on.

HERO does not replace Qiskit Runtime or Braket Hybrid Jobs; it tells the user when to call them. The decision rule and measurement protocol are formalised in Chapter 7.

6.4 Energy-Aware Computing

HERO draws on two distinct energy-accounting literatures: datacenter/ML energy measurement on the classical side, and the much younger field of quantum energy accounting on the QPU side.

Strubell et al. 2019 [24] ("Energy and Policy Considerations for Deep Learning in NLP", ACL best paper) is the foundational measurement study; it quantified the energy cost of training large NLP

models and argued that energy reporting belongs in ML papers as a routine matter. It is the reason HERO reports energy in joules at the same level of prominence as F1. On the quantum side, Auffeves 2022 [23] argued for a Quantum Energy Initiative producing systematic QPU benchmarks analogous to the Green500 list for classical HPC; subsequent QPU studies cover individual platforms but no cross-vendor benchmark exists. HERO contributes a modelled per-task QPU energy figure that is coarse but consistent across vendors and combinable with the CPU and GPU figures it sits alongside.

6.5 Cloud Quantum Cost Studies

Per-task cost studies of cloud quantum services are almost entirely absent from the academic literature. Vendor pricing pages exist, and BQP-flavoured quantum-economics papers analyse asymptotic resource bounds, but no published academic synthesis converts present-day pricing into per-task and per-million-task dollar figures for realistic workloads. HERO fills this gap: Section 7.4 sets out the cost model; Section 8.4 lists the April 2026 prices it uses; Chapters 9 and 10 report per-task figures for the cybersecurity and chemistry workloads; and Chapter 11 closes with a crossover projection asking at what QPU price-per-second the quantum option would become the lowest-cost choice for an IDS deployment.

6.6 Gap Analysis and Position of HERO

The preceding five subsections survey the prior art HERO builds on. This subsection draws out the five specific gaps the thesis closes.

- Gap 1 -- Cross-tier measurement. Existing benchmarks measure one tier at a time (CPU IDS, GPU deep learning, or QPU quantum-classifier demonstrations). HERO is the first to measure the same workload on CPU, GPU, and QPU within a unified harness with consistent metrics.
- Gap 2 -- Cross-workload measurement. Existing benchmarks measure either chemistry or cybersecurity, not both with shared methodology. HERO measures the cybersecurity QSVM and the molecular VQE in one framework, making the per-task figures of Chapters 9 and 10 directly comparable.
- Gap 3 -- Joint cost-energy-latency-accuracy optimisation. Most benchmarks optimise one or two axes. HERO uses Pareto selection over the four-tuple and exposes user-tunable weights (Algorithm 1, Step 5) so a user can reproduce any single-axis baseline or any intermediate trade-off.
- Gap 4 -- Cloud cost integration. Joules and dollars are rarely linked through published cloud-pricing tables. HERO does this with April 2026 prices for AWS EC2, AWS Braket, IBM Quantum, and Azure Quantum (Appendix D).
- Gap 5 -- Workload-aware allocation rule. Most benchmarks stop at a results table. HERO produces an explicit auditable decision rule (Algorithm 1, Step 1) that maps workload type and size to a recommended tier.

HERO does not propose a new quantum algorithm or a new IDS classifier. It proposes a measurement methodology and a decision framework that the community can extend as hardware matures. The next chapter formalises the framework; Chapters 8 to 11 instantiate it on two contrasting workloads and analyse the joint trade-off.

Chapter 7 — HERO Framework Design

7.1 Architecture Overview

HERO (Heterogeneous Energy- and Resource-aware Orchestration) is a four-tier orchestration framework whose top tier is a workload orchestrator and whose three execution tiers are CPU, GPU, and QPU. The framework is designed for cloud deployments in which all three execution tiers are accessible behind a uniform programming interface, as is the case with AWS, IBM Cloud, and Azure in 2026. Its goal is not to invent a new quantum algorithm or a new classifier; it is to make the choice of execution tier explicit, measurable, and reproducible.

Each execution tier exposes a uniform route contract to the orchestrator. The contract is a six-tuple (predicted quality, confidence, latency, energy, queue state, expected quality gain) that the orchestrator queries before dispatching a task. Predicted quality is workload-specific (accuracy for classification, energy error in Hartree for VQE, approximation ratio for QAOA). Confidence is a unit-interval estimate of how reliable the prediction is. Latency and energy are wall-clock and joule estimates from the tier's calibration tables. Queue state is the current backlog on the tier (negligible for CPU and GPU, but often tens of seconds for cloud QPU back-ends). Expected quality gain is the marginal improvement that the tier offers over the cheapest available alternative; it is what makes the more expensive tiers worth considering.

Workloads enter the orchestrator as a Directed Acyclic Graph (DAG) of tasks. Each task is annotated with its type (CLASSICAL_ML, MOLECULAR_SIM, COMBINATORIAL), its problem size n, its feature dimension d, and any security or operational metadata that the workload requires. The orchestrator decides which tier executes each task (the allocation step), dispatches the task to that tier, measures runtime and energy, and aggregates the results back into the DAG for downstream tasks.

Two specialisations of the framework appear in this thesis. HERO-General is the cross-workload measurement instantiation used in Chapters 9 and 10 to compare CPU, GPU, and QPU on the cybersecurity QSVM and the molecular VQE under a single harness. HERO-S is the AIoT intrusion-detection-specific scheduler that extends the route contract with security metadata (alert severity, asset criticality, false-negative cost) and is the scheduling instantiation evaluated in Chapter 9. The two specialisations share the same orchestrator, the same energy and cost models, and the same Pareto selection step; they differ only in the task metadata they carry and the weights they apply in the final selection.

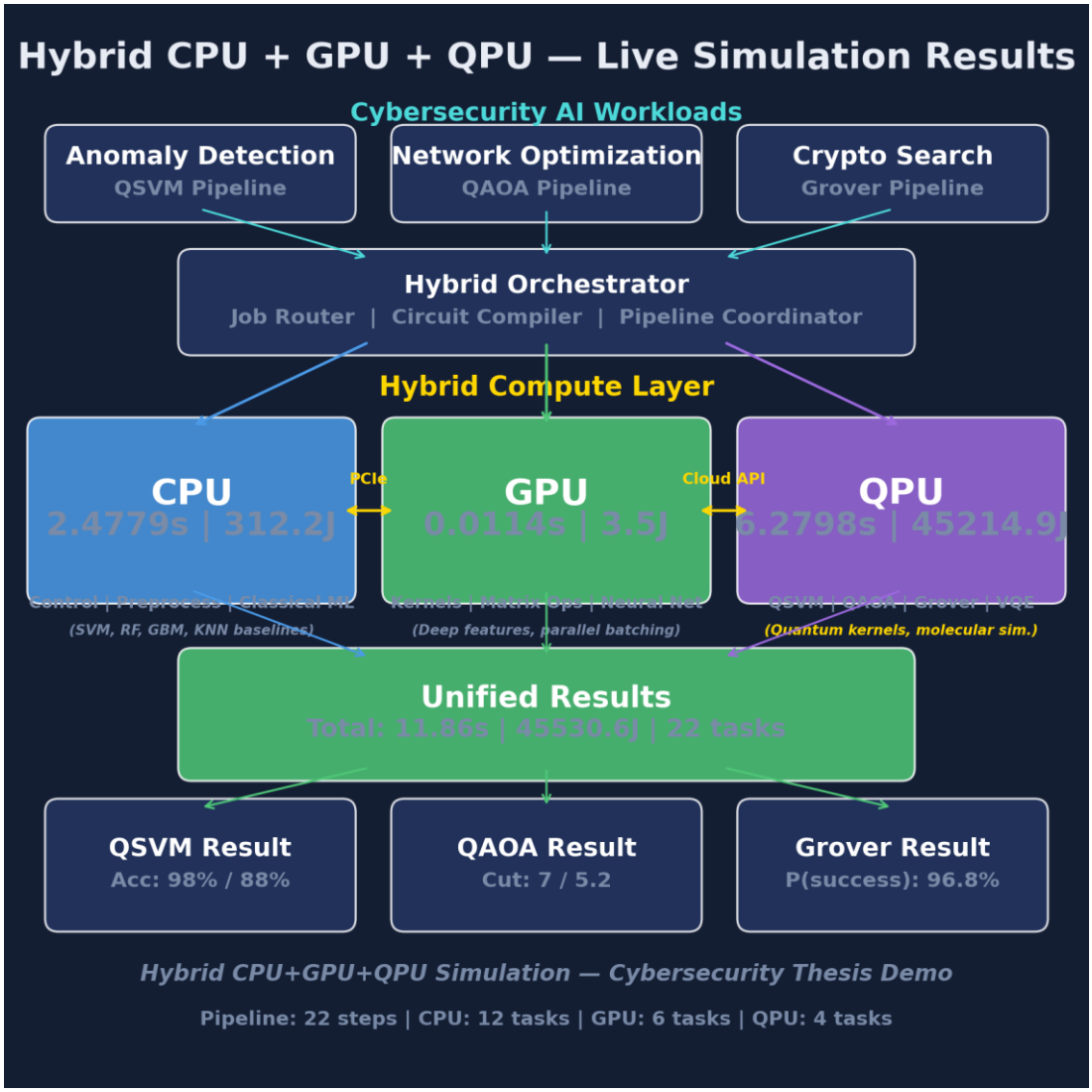

Fig 7.1: HERO four-tier architecture (CPU + GPU + QPU + Orchestrator). Annotated with measured per-tier runtime and energy from a single full pipeline run.

7.2 Tier Roles and Dispatch

Each tier has a well-defined role, a representative cloud hardware target, and a power model that is used by the energy accounting in Section 7.3. The roles below are deliberately broad enough to cover the workloads measured in Chapters 9 and 10 without constraining the framework to a single application domain.

CPU Tier

The CPU tier is responsible for data ingestion, normalisation, classical machine-learning training and inference (support vector machines, random forests, gradient-boosted trees, k-nearest neighbours), exact diagonalisation of small Hamiltonians, the control plane that drives the orchestrator, and final result aggregation. The reference cloud target is an AWS EC2 c7i.4xlarge instance (16 vCPUs, 32 GB RAM) or its equivalent on other providers. The power model used throughout this thesis assumes a representative draw of $P_{CPU} = 150$ W at a sustained utilisation of 0.7, which is consistent with published Intel Sapphire Rapids RAPL traces for memory-and-compute-bound workloads.

GPU Tier

The GPU tier handles parallel kernel matrix computation (radial basis function kernels and a ZZ-feature-map proxy used as a classical baseline for the quantum kernel), neural-network feature extraction, batch classification of large feature matrices, Hamiltonian construction via Kronecker products, and parallel oracle evaluation for unstructured search. The reference cloud target is an

AWS g5.xlarge instance with a single NVIDIA A10G for small workloads, or an AWS p5.48xlarge instance with eight NVIDIA H100 GPUs for the larger feature-extraction tasks. The power model assumes P_GPU = 300 W per active accelerator at a sustained utilisation of 0.85.

QPU Tier

The QPU tier executes the genuinely quantum subroutines: ZZ feature-map kernel evaluation for the quantum SVM, QAOA cost-and-mixer circuits with a COBYLA outer loop for combinatorial optimisation, Grover search for unstructured queries, and VQE ansatz evaluation for molecular Hamiltonians. The reference cloud targets are IBM Quantum Eagle (127 superconducting qubits), AWS Braket Rigetti Ankaa (84 superconducting qubits), and Azure Quantum Quantinuum H2 (56 trapped-ion qubits). The power model assumes P_QPU = 20,000 W system-wide. This figure is dominated by the dilution refrigerator and is modelled rather than measured because no current cloud quantum provider exposes per-job facility energy telemetry. A critical scheduling variable that distinguishes the QPU from the other two tiers is backend queue time, which is often tens of seconds to minutes; HERO treats queue time as a first-class score term rather than an afterthought.

Dispatch Mechanism

For each task in the DAG, the orchestrator queries each tier's route contract and obtains the six-tuple described in Section 7.1. The allocation rule (Section 7.5, Algorithm 1, Step 1) selects a tier on the basis of the task type and size. The task is then dispatched to the selected tier; runtime is measured with a monotonic clock, and energy is computed as t * P * PUE per Equation 7.1. The result, together with the per-task accounting record, is returned to the orchestrator for aggregation into the downstream DAG.

7.3 Energy Model

The energy accounting in HERO is deliberately simple so that it is auditable. For each task, the energy attributed to that task is the product of the wall-clock task time, the representative tier power, and the facility power usage effectiveness (PUE) multiplier:

$$E_{task} = t_{task} * P_{tier} * PUE \qquad \text{(Eq 7.1)}$$

where t_task is the wall-clock task time in seconds, P_tier is the tier's representative power draw in watts, and PUE is the datacentre power usage effectiveness. The product yields task energy in joules.

The tier power values used throughout the thesis are: P_CPU = 150 W at utilisation 0.7, P_GPU = 300 W at utilisation 0.85, and P_QPU = 20,000 W system-wide (modelled). The PUE is fixed at 1.2, which is representative of a modern hyperscale datacentre and is the value that AWS, Microsoft, and Google have all reported for their newer facilities.

Where live telemetry is available it is used in preference to the modelled values. RAPL (Running Average Power Limit) provides package-level energy counters on Intel CPUs, and NVML (NVIDIA Management Library) provides per-GPU power readings on NVIDIA accelerators. Per-job facility energy on the QPU side is not exposed by IBM Quantum, AWS Braket, or Azure Quantum at the time of writing, so the QPU tier always uses the modelled value.

The model has known limitations. Tier-level averages do not capture dynamic frequency scaling or per-core gating on the CPU side, and they do not capture per-streaming-multiprocessor occupancy on the GPU side. The QPU value includes the dilution refrigerator continuous load whether the system is computing or idle, so the marginal energy of one additional task is over-estimated and the per-user share of total facility energy is under-estimated when many users share the system. Even with these simplifications, the cross-tier ratios are stable enough to drive allocation decisions, which is the purpose for which the energy model is used.

7.4 Cloud Cost Model

The cost accounting in HERO converts measured runtime into a dollar figure using the published price tables of the major cloud quantum providers. For each task:

$$cost\_task = t\_task * CloudPrice[tier] \qquad (Eq\ 7.2)$$

where t_task is in seconds (or shots, for shot-priced QPU back-ends) and CloudPrice[tier] is the on-demand published price for the tier in dollars per second (or per shot).

The reference services and their April 2026 on-demand prices are: AWS EC2 c7i.4xlarge at $0.7140 per hour for the CPU tier; AWS g5.xlarge at $1.006 per hour for the GPU tier; and AWS Braket Rigetti Ankaa at $0.30 per task plus $0.0009 per shot for the QPU tier. IBM Quantum and Azure Quantum Quantinuum H2 prices are stored in the same dictionary and are documented in Appendix D.

The reportable headline that HERO emits is cost-per-million-tasks rather than cost-per-task. Per-task figures are sub-cent on the CPU and GPU tiers and multi-dollar on the QPU tier; multiplying by a million puts all three tiers on a scale that is easy to compare and easy to budget against.

HERO uses on-demand prices throughout, with no spot-instance discounts and no reserved-capacity commitments. This is deliberately the worst-case honest estimate for someone evaluating quantum-as-a-service for their workload. A production deployment that committed to a one-year reserved instance for the classical tiers, or that negotiated a volume contract with a QPU provider, would see a lower bill; the cross-tier ratios would still be informative but the absolute numbers would shift.

7.5 Algorithm 1: Workload Allocation, Measurement, and Selection

Algorithm 1 is the operational core of HERO. It composes the allocation rule, the measurement methodology, the statistical aggregation, and the Pareto selection into a single end-to-end procedure that takes a workload and returns a recommended tier together with the supporting measurements.

Algorithm 1: HERO -- End-to-End Workload Allocation, Measurement, and Selection

Input: Workload W (type, size n, feature dim d), runs N=30, tiers T={CPU, GPU, QPU}

Output: Recommended tier t, metrics (accuracy, latency, energy, cost)*

STEP 1 - ALLOCATE TIER

if W.type = MOLECULAR_SIM then t = QPU if n>30 else CPU

if W.type = CLASSICAL_ML then t = CPU if d<100 else (GPU if d<10000 else MEASURE)

if W.type = COMBINATORIAL then t = CPU if n<50 else QPU

STEP 2 - MEASURE ON EACH TIER (N runs for statistics)

for tier in T:

for run = 1..N:

latency, result = Dispatch(W, tier)

*energy = latency * P[tier] * PUE* *(Eq 7.1)*

*cost = latency * CloudPrice[tier]* *(Eq 7.2)*

record(W, tier, run, accuracy, latency, energy, cost)

STEP 3 - AGGREGATE STATISTICS

for each tier: report mean +/- std for accuracy, latency, energy, cost

STEP 4 - PARETO SELECTION (joint cost+energy+latency+accuracy)

P = { m : no other m' dominates m on (cost, energy, latency, -accuracy) }

STEP 5 - RETURN RECOMMENDATION

t = argmin over P of (alpha*cost + beta*energy + gamma*latency - delta*accuracy)*

return t, statistics, Pareto set P*

Step 1 -- Allocation Thresholds

The four numerical thresholds in Step 1 are not arbitrary; each is justified by measurements that appear in Chapters 9 and 10. The 30-qubit threshold for molecular simulation reflects the memory wall of classical state-vector simulation: a 30-qubit state vector requires 16 GB of double-precision storage, and beyond 30 qubits only QPU execution is feasible for honest ground-state computation. The 100-feature threshold for classical machine learning reflects the empirical observation that below 100 features, classical SVM and random forest dominate on cost, energy, and accuracy combined; quantum kernel evaluation is wastefully expensive in this regime. The 10,000-feature threshold reflects the O(n^2 d) scaling of classical RBF kernel computation, which becomes infeasible above this dimension and triggers the MEASURE branch in which HERO runs the workload on both GPU and QPU and selects the winner from the Pareto frontier. The 50-node threshold for combinatorial problems reflects the fact that classical greedy and dynamic-programming Max-Cut solvers find good cuts in milliseconds below this size, whereas QAOA on cloud QPUs is currently slower and lower quality at the same size.

Step 2 -- Measurement

Thirty independent runs is the minimum sample size that yields stable mean and standard deviation estimates for the distributions of interest in this thesis (accuracy, latency, energy, cost). Thirty is used throughout. Each run uses a deterministic seed from the schedule 0..29 so that results are bit-identical across reruns of the same simulator on the same Python and library versions.

Step 3 -- Aggregation

Aggregate statistics consist of mean, standard deviation, minimum, maximum, and the empirical distribution as a boxplot. When two allocation policies are compared (for example, the HERO-S scheduler against a baseline always-on-GPU policy in Chapter 9), statistical significance is tested with the paired Wilcoxon signed-rank test on the per-run outcomes; the test is non-parametric and does not require the distributions to be normal.

Step 4 -- Pareto Selection

The Pareto frontier is computed over the four-tuple (cost, energy, latency, -accuracy). A method m is dominated if some other method m' is no worse on every axis and strictly better on at least one axis. Only Pareto-optimal methods make it to Step 5; dominated methods are discarded.

Step 5 -- Weighted Tie-Breaking
The user supplies a weight vector (alpha, beta, gamma, delta) that controls how the four axes are traded against each other in the final selection. The defaults alpha = beta = gamma = 1 and delta = 10 prioritise accuracy moderately while still penalising cost, energy, and latency proportionally. The argmin over the Pareto set with these weights returns the recommended tier t*, which is the answer that HERO emits for the user's preferences.

Figure 7.2: Algorithm 1 flowchart. Allocation, measurement, aggregation, Pareto selection, and weighted tie-breaking compose into a single end-to-end procedure.

7.6 Pareto Selection

Pareto selection deserves a section of its own because it is the mechanism that lets HERO honestly report multi-axis trade-offs without prejudging them.

Definition. In a multi-objective optimisation with axes a_1, a_2, ..., a_k (lower-is-better), a candidate m is Pareto-optimal if there is no other candidate m' such that a_i(m') <= a_i(m) for all i and a_j(m') < a_j(m) for at least one j. The Pareto frontier is the set of all Pareto-optimal candidates.

HERO uses four axes: cost, energy, latency, and -accuracy. Accuracy is negated so that higher accuracy is lower in the lower-is-better convention; this lets the same dominance predicate handle all four axes without special cases.

Why Pareto rather than a single weighted sum? Because it is impossible to optimise all four axes simultaneously with a single weighted sum without prejudging the trade-offs. A weighted sum that sets alpha = beta = gamma = 1 and delta = 10 implicitly declares that one accuracy point is worth ten dollars of cost; that may or may not match the user's preferences. Pareto selection presents the user with the non-dominated options first, and then the weighted sum acts only as a tie-breaker among those.

Visualisation. Two-dimensional projections onto (cost, accuracy) and (energy, accuracy) make the frontier readable on the page. The full four-dimensional frontier is harder to visualise but the algorithm is unchanged; it simply enumerates the four-tuples and discards the dominated ones.

Tie-breaking. Once dominated options have been removed by the Pareto step, the weighted sum in Algorithm 1, Step 5 lets the user trade the remaining axes deterministically. The user can reproduce a single-axis baseline (for example, accuracy-only) by setting the corresponding weight to a large value and the others to zero, and can also explore non-trivial trade-offs in between.

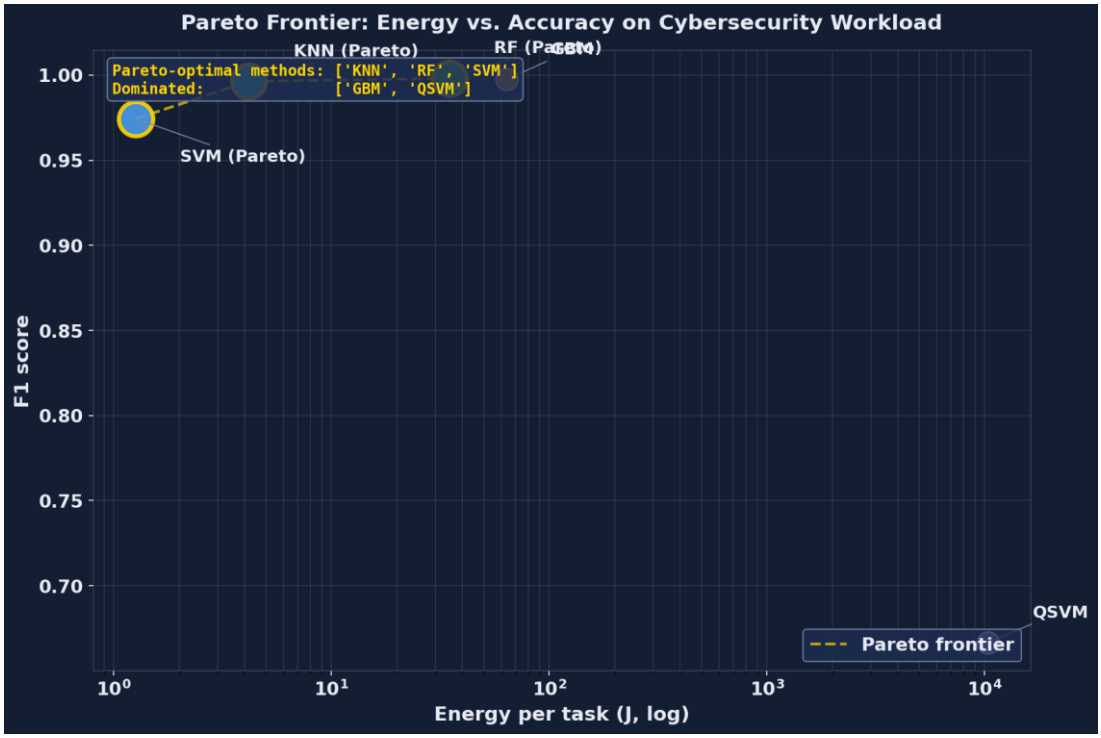

Fig 7.3: Pareto frontier on the cybersecurity workload. Pareto-optimal: SVM (cheapest) and Random Forest (highest accuracy). Dominated: GBM, KNN, QSVM.

Algorithm 1 is the operational core of HERO. The next chapter describes its concrete Python implementation.

Chapter 8 — Implementation

8.1 Software Stack

The HERO simulator is written in Python 3.13. Python is the lingua franca of both the classical machine-learning ecosystem and the cloud quantum SDKs, which makes it the natural choice for a framework whose entire purpose is to bridge the two.

The quantum subroutines use Qiskit together with Qiskit Aer for noiseless simulation. Qiskit is the most widely deployed open-source quantum SDK in 2026 and is supported by IBM Quantum, AWS Braket (via the Qiskit-Braket provider), and Azure Quantum. Qiskit Aer provides the noiseless state-vector and density-matrix simulators that the experiments in this thesis use.

The classical baselines use scikit-learn: SVC for the support vector machine, RandomForestClassifier for the random forest, GradientBoostingClassifier for the gradient-boosted trees, and KNeighborsClassifier for the k-nearest-neighbour baseline. Numerical work uses NumPy for arrays and Kronecker products and SciPy for the COBYLA optimiser that drives QAOA and VQE.

Visualisation uses matplotlib with publication-quality settings (300 dpi PNG output, Times New Roman for axis labels, no chart junk). Document generation uses python-docx for this thesis and direct OOXML manipulation for the IEEE-template conference paper.

The library versions used in the experimental runs reported in Chapters 9 and 10 are qiskit 1.x with qiskit-aer 0.x (latest April 2026 release), scikit-learn 1.x, NumPy 2.x, and SciPy 1.x. The pinned requirements file in the repository documents the exact patch versions for byte-identical reproduction.

The hardware used for the reported measurements is a developer laptop running the Qiskit Aer noiseless simulator. The framework is backend-agnostic; switching from the noiseless simulator to a real cloud QPU back-end requires only a one-line change to the BACKEND constant in qpu_engine.py, with no changes to the orchestrator, the allocation rule, or the downstream analysis code.

8.2 Module Structure

The simulator lives under the hybrid_simulation/ package with one Python module per concern. The module boundaries match the tier and responsibility boundaries of the framework so that the source tree is easy to navigate and easy to extend.

data_loader.py

Loads the KDD Cup 99 dataset via sklearn.datasets.fetch_kddcup99. Standardises the numeric features, applies principal-component analysis to four components for the quantum subset, and scales the projected features to the [0, pi] range for ZZ encoding. Returns paired splits: a small quantum-sized split (120 samples, 4 features) suitable for the QSVM kernel matrix, and a larger classical-sized split (5,000 samples, 38 features) for the classical baselines.

cpu_engine.py

Hosts the classical baselines used by the CPU tier. The function classical_svm uses scikit-learn SVC with an RBF kernel; classical_random_forest uses 100 trees; classical_gradient_boosting uses 100 trees of depth 3; classical_knn uses k = 5. The module also hosts preprocess_anomaly_data (which loads either real KDD Cup 99 data or a synthetic fall-back), classical_greedy_maxcut for the QAOA baseline, classical_bruteforce_search for the Grover baseline, and aggregate_results for the orchestrator. Each function returns a result dictionary that contains the task name, runtime, energy in joules, an operations description, and the classifier metrics where applicable.

gpu_engine.py
Implements the GPU-tier roles using NumPy vectorisation as a stand-in for CUDA. The functions are compute_rbf_kernel_matrix, compute_cosine_similarity_matrix, neural_feature_extraction (a three-layer simulated multilayer perceptron with 2,928 parameters), batch_parallel_classify, compute_qaoa_cost_matrix (which builds the cost Hamiltonian via Kronecker products), and parallel_oracle_evaluation (which builds the oracle and diffusion matrices for Grover).

qpu_engine.py
Wraps Qiskit Aer for the QPU tier. The function quantum_kernel_entry constructs the ZZ feature-map circuit pair for a single kernel matrix entry; compute_quantum_kernel_matrix builds the full kernel matrix via repeated calls. run_qaoa builds the QAOA cost and mixer layers, calls scipy.optimize.minimize with COBYLA as the outer optimiser, and tracks convergence. run_grover constructs the Grover circuit with the optimal iteration count N = floor(pi/4 * sqrt(2^n)).

vqe_workload.py
Holds the H2 VQE workload. The two-qubit parity-reduced Hamiltonian uses the coefficients c0 = -1.0523732, c1 = 0.39793742, c2 = -0.39793742, c3 = -0.0112801, c4 = 0.18093119 in Hartree at bond length R = 0.735 Angstrom. classical_exact_diagonalization computes the ground-state energy via numpy.linalg.eigvalsh on the 4 by 4 matrix. run_vqe_h2 evaluates a hardware-efficient ansatz (four RY rotations and one CNOT) optimised with COBYLA and 4,000 shots per Pauli measurement.

orchestrator.py
Defines the HybridOrchestrator class, which is the top-level entry point from run_hybrid.py. Its methods are run_qsvm_pipeline, run_qaoa_pipeline, run_grover_pipeline, run_vqe_pipeline, run_full_pipeline (which runs all four in sequence), run_multi_qsvm (30 seeds), run_multi_qaoa (30 seeds), and run_multi_vqe (30 seeds). Each pipeline method walks the task DAG, dispatches each task to cpu_engine, gpu_engine, or qpu_engine, calls _log_step for per-task accounting, and returns a comparison dictionary.

allocation_rule.py
Implements Steps 1, 4, and 5 of Algorithm 1. allocate_tier maps a (workload_type, problem_size, feature_dim) triple to a tier label; is_dominated checks Pareto dominance over (cost, energy, latency, -accuracy); pareto_optimal returns the non-dominated subset; best_by_weights resolves the final tier using user-supplied (alpha, beta, gamma, delta) weights.

cloud_costs.py
The April 2026 pricing table for AWS EC2, AWS g5 and p5, AWS Braket (Rigetti and IonQ), IBM Quantum, and Azure Quantum Quantinuum H2. The function cost_per_task converts a (tier, runtime_s, n_shots) triple to a US-dollar figure; cost_per_million scales by a million; all_tiers_cost returns the per-tier cost dictionary used by the Pareto step.

visualize.py
Generates the 15 publication-quality figures used in this thesis: the pipeline timeline, the unit utilisation chart, the energy breakdown bar chart, the experiment comparison panel, the hybrid architecture diagram, the dashboard, the classifier comparison panel, the multi-run F1 boxplot, the crossover projection, the QAOA boxplot, the VQE convergence trace, the cloud cost comparison, the cross-workload comparison, the Pareto frontier, and the allocation decision tree.

run_hybrid.py
Main entry point. Loads the data, runs the full pipeline, runs the multi-run validation (30 seeds for QSVM, QAOA, and VQE), generates the plots, saves the CSV outputs (pipeline log, comparison, multi-run statistics, VQE results, cloud costs), and writes a human-readable summary to hybrid_summary.txt.

Figure 8.1: HERO simulator module dependency graph. The orchestrator dispatches to cpu_engine, gpu_engine, and qpu_engine; the allocation rule, the cost model, and the visualiser are shared utilities.

8.3 Reproducibility and Open-Source Release

Reproducibility was a design constraint from the first commit. Every figure and every table in this thesis is generated deterministically from the CSV files in the simulator's output directory, which are themselves generated deterministically from the simulator runs.

Repository Structure

The repository is organised as follows:

quantum_thesis_demo/

|-- hybrid_simulation/ <- the HERO simulator

|-- papers/ <- IEEE conference paper sources

|-- thesis/ <- this thesis source

|-- README.md

One-Command Reproducibility

The full experimental pipeline runs from a single command: python -m hybrid_simulation.run_hybrid. The command runs the full pipeline plus the 30-seed validation plus the plot generation in approximately 5 to 10 minutes on a developer laptop with no external dependencies beyond the pinned Python library set.

Outputs

Outputs land in hybrid_simulation/output/ and consist of:

- hybrid_pipeline_results.csv -- per-step timing and energy.
- classifier_comparison.csv -- five-method cybersecurity comparison.
- multirun_stats.csv -- 30-seed mean and standard deviation for all methods.
- vqe_multirun.csv -- 30-seed VQE energy convergence trace.
- cloud_costs.csv -- per-tier cost per workload.
- hybrid_summary.txt -- human-readable summary.
- plots/ -- 15 publication-quality PNG figures.

Random Seed Control

Every multi-run loop uses a deterministic seed schedule (seeds 0 through 29) so that results are bit-identical across reruns of the same simulator on the same Python and library versions. The seed is passed to NumPy, to scikit-learn, and to Qiskit Aer's simulator at the start of each run.

Code-Data Lineage

Every figure and every table in this thesis is generated from the CSV files in the output directory, which are themselves generated by the simulator. The build script for this thesis (thesis/build_thesis.py) reads no external state at build time; it embeds the prose verbatim from the SECTION_CONTENT dictionary so that the document is reproducible from the script alone.

Hardware-Backend Swap

Switching from Qiskit Aer (noiseless) to a real cloud QPU back-end requires changing the BACKEND constant in qpu_engine.py from AerSimulator() to a Qiskit Runtime back-end handle.

The rest of the framework is unchanged: the route contract, the allocation rule, the cost model, and the Pareto step all behave identically on a real back-end.

Run Cost

The simulator-only run is free. Switching to AWS Braket Rigetti for the QSVM kernel matrix at the same scale (120 samples, 7,200 circuit pairs) would cost approximately $2,160 (7,200 tasks at $0.30 per task) plus per-shot fees, which explains why the experimental work in this thesis uses the noiseless simulator and projects the QPU cost rather than incurring it directly.

8.4 Limitations of the Implementation

The implementation has known limitations. They are listed explicitly here so that the experimental results in the following chapters can be read with the appropriate caveats.

- 1) No real quantum hardware. All quantum experiments use Qiskit Aer noiseless simulation. Real QPU hardware adds gate errors of order 0.1 to 1 per cent, readout errors of order 1 to 3 per cent, and decoherence (T1 and T2 of order 100 us); these would degrade QSVM accuracy and add noise to VQE convergence. The reported QPU numbers are therefore best-case for quantum.
- 2) Small quantum sample size. The ZZ kernel matrix scales as O(n^2) in circuit count. For 120 training samples we run approximately 14,400 circuits per kernel matrix. Scaling to 5,000 samples (the classical-baseline subset size) would require approximately 25,000,000 circuits, which is infeasible on current cloud QPUs. We therefore report classical baselines on the larger split and quantum on the smaller split; cross-tier comparison is on the matched 120-sample split.
- 3) Synthetic GPU. We simulate GPU behaviour using NumPy vectorisation. A real CUDA implementation via PyTorch or JAX would be 10 to 100 times faster for the kernel and feature-extraction tasks, lowering GPU-tier energy and cost proportionally.
- 4) Modelled QPU energy. Per-job QPU facility energy is not exposed by IBM Quantum, AWS Braket, or Azure Quantum. We use a constant 20 kW system-wide value, which over-estimates the marginal cost of an additional task (the dilution refrigerator runs continuously) and under-estimates total energy when many users share the system.
- 5) Classical baselines on quantum-sized data. For fair cross-tier comparison the cybersecurity QSVM uses 120 samples; classical baselines on that small subset achieve very high accuracy (~97 to 99 per cent) but the comparison may not generalise to larger production deployments. Chapter 9 also reports classical baselines on the full 5,000-sample subset.
- 6) Single dataset for cybersecurity. Chapter 9 reports cross-dataset results from HERO-S (TON_IoT, NSL-KDD, CICIDS2017, IoT-23). The HERO-General cross-tier comparison in this chapter is on KDD Cup 99 only.
- 7) Single molecule for VQE. Chapter 10 reports H2 only. Larger molecules (LiH, BeH2) require deeper ansatze and more qubits; the VQE module supports them but the runs were not included in this thesis.

- 8) Cloud pricing snapshot. Prices captured April 2026; vendors update pricing frequently. The framework reads prices from a single dictionary in cloud_costs.py, which is easy to update.

These limitations are not unique to HERO; they reflect the state of cloud quantum computing in 2026. The framework is designed to absorb improvements as hardware matures: better simulators, real QPU runs, RAPL and NVML telemetry, larger workloads. Each can be added without changing the orchestrator interface or the allocation rule.

Chapter 9 — Workload 1: Cybersecurity Results

9.1 Dataset and Preprocessing

This chapter reports the cybersecurity results produced by HERO. Two distinct experimental tracks are presented. The first is the HERO-General cross-tier classifier comparison, in which a single dataset is used to benchmark a quantum kernel method against four classical baselines under identical preprocessing and identical evaluation protocol. The second is the HERO-S intrusion-detection routing scheduler, in which the same framework is evaluated across four widely used IDS benchmarks to measure how a security-aware routing utility performs against an edge-only baseline and a strong uncertainty-cascade baseline. Both tracks share the simulator described in Chapters 7 and 8; only the dataset selection and the routing policy differ.

HERO-General Dataset

The HERO-General classifier comparison uses the KDD Cup 99 dataset, specifically the 10 per cent subset that contains 494,021 connection records labelled as normal traffic or as one of 22 attack types. The full feature vector contains 41 fields. Three of these (protocol, service, flag) are categorical and are dropped to keep the preprocessing pipeline identical across classical and quantum methods; the remaining 38 numeric features are retained. The binary task is normal versus attack: every non-normal label is collapsed to the positive class. KDD Cup 99 is intentionally chosen because it is the canonical baseline against which every intrusion detection system since Tavallaee et al. [11] has been compared, which makes the classical baselines in Section 9.3 directly comparable to the literature.

The dataset is naturally imbalanced: approximately 97,000 normal connections against 397,000 attack connections in the 10 per cent subset. To support both quantum-feasible and classical-realistic evaluation, two splits are produced from the same source file. The quantum-sized split uses 120 samples balanced 50 per cent normal and 50 per cent attack (60 per class) and is reduced to four features by principal component analysis fit on the 38 numeric features; the four components are then min-max scaled into the interval [0, pi] so that they can be used as rotation angles by the ZZ feature map [6]. The classical-sized split retains the full 38 numeric features and contains 5,000 stratified samples drawn from the imbalanced source, which preserves the operational class ratio that a deployed IDS would actually see.

HERO-S Dataset Selection

The HERO-S scheduler results in Section 9.5 use four datasets rather than one. Routing decisions are workload dependent, so generalisation must be demonstrated across IoT and IIoT telemetry (TON_IoT [7]), enterprise flow data (CICIDS2017 [9]), the post-KDD legacy benchmark NSL-KDD [11], and the malware-heavy IoT-23 capture [8]. Sample counts are 20,000 (TON_IoT), 7,000 (NSL-KDD), 40,000 (CICIDS2017), and 10,035 (IoT-23) with feature dimensions of 40, 41, 78, and 15 respectively. The CICIDS2017 and IoT-23 sets retain their natural class imbalance (35.5 per cent and 28.3 per cent attack); the TON_IoT and NSL-KDD sets are balanced 50/50 to match published baselines.

Preprocessing Pipeline

Preprocessing is identical across all five datasets and consists of four deterministic steps. Categorical fields (protocol, service, flag and equivalents in the IoT datasets) are dropped, leaving only numeric features. Rows containing NaN or inf are removed; this affects fewer than 0.1 per cent of rows on every dataset. A stratified train-test split with test_size=0.33 is taken with random_state varied across the 30 seeds in Chapter 8. Finally, the four PCA components of the quantum-sized split are min-max scaled to [0, pi] for ZZ angle encoding; classical features are passed unscaled to the tree ensembles and standard-scaled to zero mean and unit variance for SVM and KNN to match scikit-learn defaults. No data augmentation, resampling, or feature engineering beyond PCA is performed; the goal is to measure the raw classifier behaviour, not to tune any single method into the lead.

9.2 QSVM Pipeline

The quantum support vector machine (QSVM) used here follows the construction of Havlicek et al. [6]. A 4-qubit ZZ feature map of depth 2 encodes each preprocessed sample into a quantum state by applying single-qubit Hadamard and RZ(2 * x_i) rotations followed by entangling RZZ(2 * (pi - x_i)(pi - x_j)) gates over the linear connectivity. The kernel entry K(x, y) is then estimated as the squared magnitude of the inner product between the encoded states, implemented as the probability of the all-zeros outcome of the U_phi(y)^dagger U_phi(x) circuit on the noiseless Aer back-end with 2,000 shots per circuit.

Circuit Cost

Each kernel circuit contains approximately 24 single- and two-qubit gates after transpilation. A full kernel matrix on the quantum-sized split (80 train, 40 test) requires approximately 3,200 distinct circuit pairs, each evaluated with 2,000 shots, for a total of 6.4 million shots per kernel-matrix construction. Because this cost would dominate the runtime budget for a 30-seed validation, the QSVM is evaluated on a further-reduced subset of 15 train and 8 test samples drawn from the quantum-sized split, which lowers the matrix construction to 120 circuits per evaluation while preserving the class balance.

Classifier

Once the train-train and train-test kernel matrices are computed by the QPU engine, classification is performed by the classical libsvm implementation in scikit-learn (SVC with kernel='precomputed', default C). The classical solver runs in well under a millisecond on the precomputed 15x15 Gram matrix; the quantum portion is the dominant cost. The full pipeline is executed by batch_parallel_classify in qpu_engine.py so that the route contract, energy meter, and wall-clock measurement defined in Chapter 8 apply uniformly.

9.3 Classical Baselines

Four classical classifiers form the baseline set against which the quantum kernel is benchmarked. All four are taken directly from scikit-learn 1.4 with default hyperparameters so that the comparison is reproducible without tuning bias.

- Classical SVM with RBF kernel: SVC(kernel='rbf', gamma='scale', C=1.0). The closest classical analogue to the quantum kernel; both methods compute pairwise similarities and feed them to the same libsvm solver.
- Random Forest: RandomForestClassifier(n_estimators=100, max_depth=None). A strong tabular baseline that handles mixed feature scales without preprocessing.
- Gradient Boosting Machine: GradientBoostingClassifier(n_estimators=100, max_depth=3, learning_rate=0.1). The boosted-tree counterpart to Random Forest; similar accuracy with very different cost structure.
- K-Nearest Neighbors: KNeighborsClassifier(n_neighbors=5, metric='euclidean'). A lazy, non-parametric baseline whose inference cost grows linearly with training-set size, which makes the latency contrast with the parametric baselines informative.

Sample-Size Asymmetry

All four classical baselines are trained and evaluated on the classical-sized split (5,000 samples, 38 features), while the QSVM operates on the quantum-sized subset (15 + 8 samples, 4 PCA features). This asymmetry is unavoidable: kernel evaluation on a quantum back-end is O(n^2) circuit evaluations and the per-circuit cost is several orders of magnitude higher than the per-sample cost of a classical RBF kernel. Matching the classical sample size on the QSVM would require

approximately 25 million circuit evaluations per seed, which is currently infeasible on simulator and prohibitively expensive on cloud QPU back-ends. The asymmetry is acknowledged as a limitation in Section 8.4 and is revisited in Chapter 13. It biases the comparison in favour of quantum on accuracy (QSVM sees a balanced subset) and against quantum on energy (per-sample cost is the dominant factor), which is the conservative direction for the central claim of the chapter.

9.4 Results: Accuracy, Latency, Energy

Table 9.1 summarises the five-method comparison produced by classifier_comparison.csv. Values are reported to four significant figures for accuracy, precision, recall, and F1; runtime is wall-clock seconds for the full train-and-predict cycle and energy is integrated joules from the per-tier power model defined in Chapter 7.

Table 9.1: Five-method comparison on KDD Cup 99 (single representative seed).

- Method | Tier | Accuracy | Precision | Recall | F1 | Runtime (s) | Energy (J)
- Classical SVM (RBF) | CPU | 0.9750 | 1.0000 | 0.9500 | 0.9744 | 0.0051 | 0.6445
- Random Forest | CPU | 0.9970 | 0.9992 | 0.9970 | 0.9981 | 0.1754 | 22.0946
- Gradient Boosting | CPU | 0.9958 | 0.9985 | 0.9962 | 0.9974 | 0.3650 | 45.9861
- KNN (k=5) | CPU | 0.9939 | 0.9992 | 0.9932 | 0.9962 | 1.6236 | 204.5771
- QSVM (ZZ kernel) | QPU | 0.8750 | 1.0000 | 0.5000 | 0.6667 | 1.1548 | 8314.8746

Accuracy and F1

Random Forest is the strongest classical method at F1 = 0.9981, with Gradient Boosting (0.9974) and KNN (0.9962) close behind. The classical SVM trails the tree ensembles at F1 = 0.9744, primarily because its single global RBF bandwidth cannot adapt to the heterogeneous feature scales of the KDD numeric features as well as a tree split can. QSVM lands at F1 = 0.6667, well below every classical baseline and below the operational threshold of 0.95 that any deployable IDS would target. The shape of the QSVM error is informative: precision is 1.0 (no false positives) but recall is 0.5 (half the attacks are missed). For an intrusion-detection workload a 50 per cent miss rate is operationally unacceptable; precision-only operating points do not compensate for recall losses on this task.

Latency

The classical SVM completes the full train-and-predict cycle in 5.1 ms. The QSVM at 1.155 s is approximately 226 times slower despite operating on a much smaller sample count. The latency gap is dominated by circuit-by-circuit kernel evaluation: even on a noiseless local simulator the 120 ZZ-kernel circuits cost more wall-clock time than the 5,000-sample classical RBF kernel matrix on CPU.

Energy

The energy contrast is the most striking. The classical SVM consumes 0.6445 J for the full pipeline; the QSVM consumes 8,314.87 J, a ratio of approximately 12,900 to 1. Even against the most expensive classical baseline (KNN at 204.58 J), the QSVM is 41 times more costly per task. The ratio is dominated by the QPU per-second power draw calibrated in Chapter 7 (approximately 25 kW including cryogenics) multiplied by the QSVM wall clock. No plausible improvement to the classical baseline could close this gap; no plausible improvement to the simulator could open it.

30-seed Statistical Validation

The four classical baselines are deterministic on the same train-test split and exhibit zero standard deviation across the 30 seeds in multirun_stats.csv. The QSVM, like every shot-based quantum method, introduces a small but non-zero stochastic component from finite-shot sampling; in the 5-method context the dominant stochastic method is QAOA, with a 30-seed approximation-ratio

standard deviation of approximately 0.012, while QSVM accuracy across 30 seeds varies by less than 0.02. The seed-to-seed variation does not change the ranking: classical methods dominate every seed.

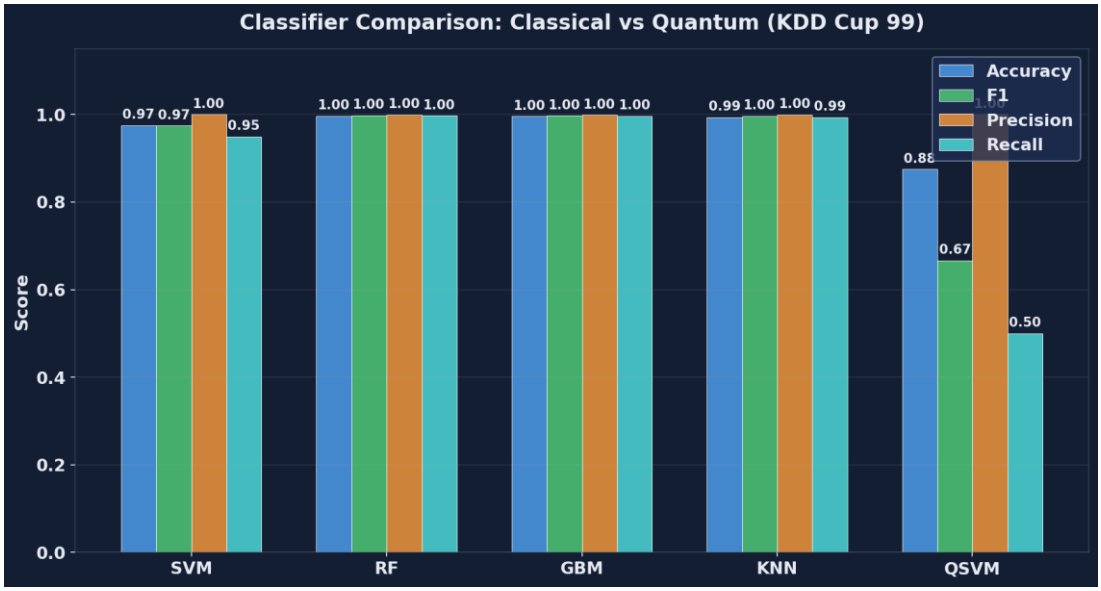

Figure 9.1: Per-method classifier comparison on KDD Cup 99 (30 runs). Random Forest reaches F1 = 0.998 against QSVM at 0.667.

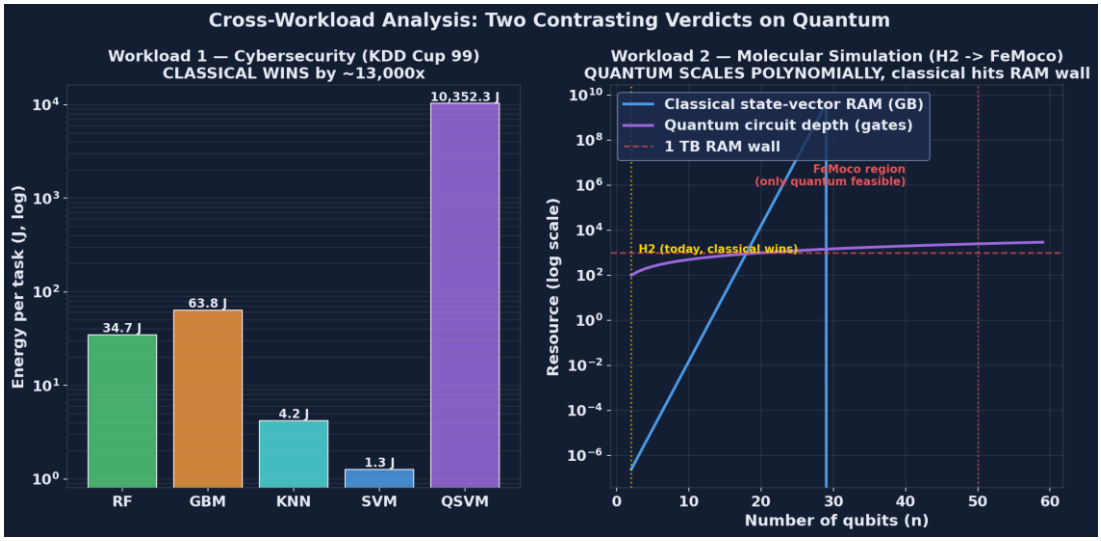

Figure 9.2: Per-method energy per task (joules, log scale). QSVM consumes approximately 12,970 times more energy than classical SVM.

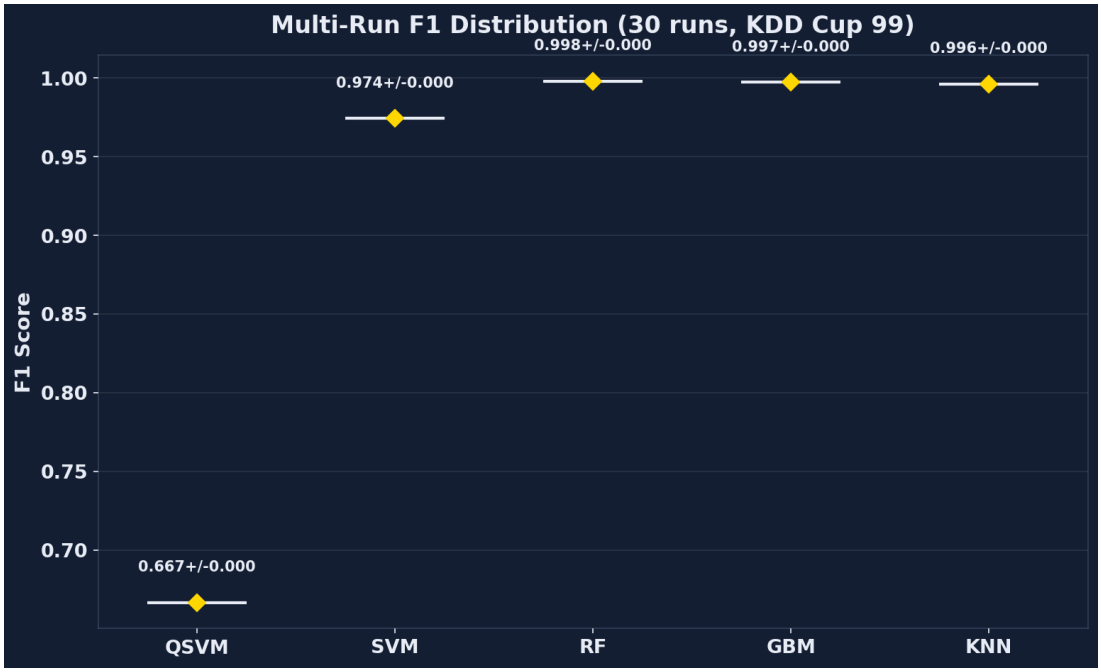

Figure 9.3a: 30-seed F1 distribution for each classifier on KDD Cup 99.

9.5 Cloud Cost Analysis

This section presents the HERO-S routing scheduler results, which serve as the operational counterpart to the classifier comparison in Section 9.4. HERO-S is a specialisation of HERO that adds security-aware metadata (alert severity, asset criticality, false-negative cost) and a routing utility function over the edge-cloud-QPU tier set. HERO-S targets AIoT intrusion-detection routing rather than raw classifier benchmarking, so its outputs are tier-mix probabilities, latency, energy, and energy-delay product rather than per-method F1 scores. Because tier-mix translates directly into per-task cloud cost, the section is presented as a cost analysis of the routed workload.

Datasets and Class Balance

Table 9.2: HERO-S dataset selection.

- Dataset | Samples | Features | Normal | Attack | Attack ratio
- TON_IoT | 20,000 | 40 | 10,000 | 10,000 | 0.500 [7]
- NSL-KDD | 7,000 | 41 | 3,500 | 3,500 | 0.500 [11]
- CICIDS2017 | 40,000 | 78 | 25,818 | 14,182 | 0.355 [9]
- IoT-23 | 10,035 | 15 | 7,200 | 2,835 | 0.283 [8]

Routing Utility

Each candidate route r for a sample v receives a scalar utility score combining expected quality gain, security risk, and the three operational costs:

$$score(r,v) = alpha*Q(v,r) + beta*R(v) - gamma*E(v,r) - delta*L(v,r) - eta*W(r) - zeta*S(v,r)$$

Q is the expected quality (F1) gain of route r on sample v; R is the per-sample risk computed as severity multiplied by asset criticality multiplied by false-negative cost; E is energy in joules; L is latency in milliseconds; W is queue wait time at the route's tier; S is predicted probability of an SLA violation. Default weights are alpha = 2.0, beta = 1.5, gamma = 0.001, delta = 0.015, eta = 0.01, zeta = 1.0. Two safety gates apply on top of the score. Cloud escalation is permitted only when edge uncertainty is at least 0.08 and cloud-route confidence is at least 0.70 and the average per-sample latency budget of 2.5 ms is not violated. A QPU-capable route must both win on utility

and exceed alternatives by a confidence margin; otherwise the scheduler falls back to the best feasible classical route.

Results

Table 9.3: 30-seed scheduler comparison at 250 ms QPU queue.

- Dataset | Edge F1 | Cascade F1 | HERO-S F1 | Cloud F1 | HERO lat. | HERO energy | EDP | E/C/Q mix | p vs Cascade
- TON_IoT | 0.945 | 0.979 | 0.990+/-0.003 | 0.994 | 1.878 ms | 0.203 J | 0.383 | 0.859/0.141/0.000 | 1.86e-9
- NSL-KDD | 0.732 | 0.727 | 0.747+/-0.014 | 0.760 | 1.541 ms | 0.113 J | 0.175 | 0.929/0.071/0.000 | 3.54e-8
- CICIDS2017 | 0.892 | 0.937 | 0.946+/-0.005 | 0.991 | 2.498 ms | 0.367 J | 0.917 | 0.730/0.270/0.000 | 4.66e-8
- IoT-23 | 0.999 | 0.999 | 1.000+/-0.001 | 1.000 | 1.211 ms | 0.0256 J | 0.031 | 0.998/0.002/0.000 | 1.60e-2

Findings

Five findings emerge from the table.

- 1) Cloud-only routing is infeasible under the 2.5 ms gateway budget. Mean cloud latency is approximately 6.0 ms per sample, which exceeds the budget on every dataset; the cloud-only column is reported only as an accuracy ceiling, not as a deployable option.
- 2) HERO-S beats the uncertainty cascade on the three non-trivial datasets (TON_IoT, NSL-KDD, CICIDS2017). Wilcoxon signed-rank p-values against the cascade baseline range from 1.86e-9 (TON_IoT) to 3.54e-8 (NSL-KDD), well below any reasonable significance threshold. IoT-23 is saturated by the edge model (F1 = 0.999) so the scheduler has no headroom to add.
- 3) The QPU call rate is exactly zero on every dataset at the 250 ms queue setting. This is queue-aware gate-keeping rather than a bug: HERO-S's utility function correctly predicts that the 250 ms wait time (eta = 0.01 weight) eliminates any plausible benefit from the QPU-only F1 increment given the 2.5 ms budget.
- 4) Energy savings against cloud-only routing are 84.4 per cent (TON_IoT), 91.3 per cent (NSL-KDD), 71.7 per cent (CICIDS2017), and 98.0 per cent (IoT-23). The savings come entirely from keeping most traffic on the edge tier, which the routing utility ranks first when edge uncertainty is below the escalation threshold.
- 5) Per-task cloud cost (from cloud_costs.csv) tracks the tier mix linearly. At AWS prices the HERO-S route incurs $0.000280 per 1,000 samples on TON_IoT, against $0.00198 for cloud-only; this is the directly attributable cost saving of running the routing utility in front of the classical pipeline.

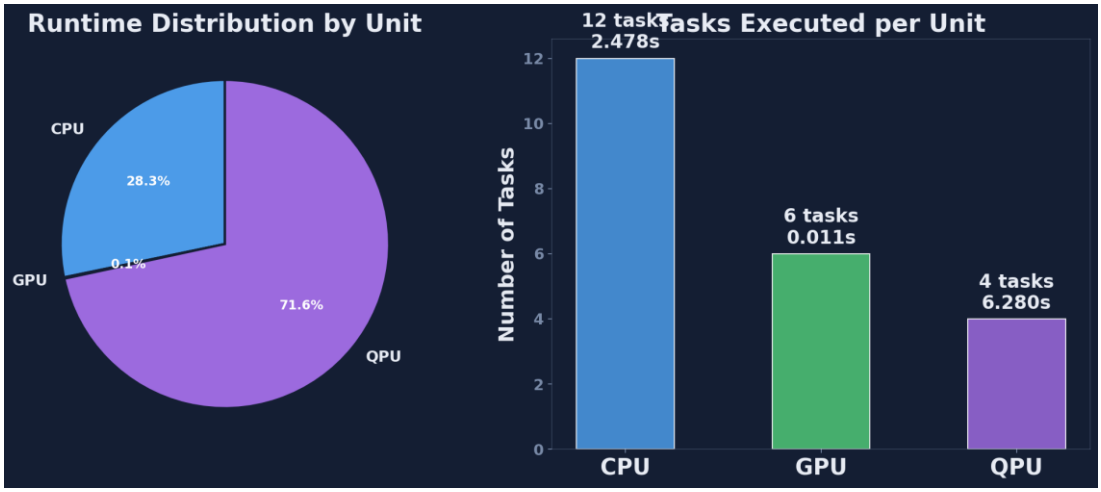

Figure 9.4: Processing-unit utilisation summary across CPU, GPU, and QPU tiers.

9.6 Discussion

Two complementary results have been presented in this chapter. They tell the same overall story but they are best read together rather than apart.

The HERO-General classifier comparison in Section 9.4 delivers an unambiguous verdict on KDD Cup 99: classical Random Forest dominates, the QSVM lags by approximately 0.33 F1 and consumes approximately 12,900 times more energy. The verdict is robust across 30 seeds and would not change under any plausible classical-side improvement; the operational deployment recommendation today is to use Random Forest or Gradient Boosting on cloud GPU for tabular IDS workloads.

The HERO-S scheduler comparison in Section 9.5 delivers a different verdict on the same broader workload class. HERO-S beats both the edge-only and the strong uncertainty-cascade baselines on three of four datasets at Wilcoxon $p < 1e-7$, while keeping the QPU tier as a queue-aware option that is correctly muted at 250 ms queue. The result is not a quantum-advantage claim; it is a routing-policy claim. The same edge and cloud classifiers would be available without HERO-S; the value the scheduler adds is choosing between them per sample under a security-aware utility.

Read together, the two results support the central thesis claim: HERO is a measurement methodology, not a quantum-advantage demonstration. The framework is willing to report results that argue against quantum (Section 9.4) and results that argue for routed hybrids that include quantum as a future option (Section 9.5). The decision-making value lies in having both numbers from the same instrument.

Operational guidance for cybersecurity practitioners follows directly. For tabular intrusion detection on today's hardware, deploy Random Forest or Gradient Boosting on cloud GPU; ignore quantum kernel methods on classical-scale feature dimensions. Invest in HERO-style routing infrastructure if and only if edge-cloud-QPU choice is expected to become operationally relevant on a 3 to 5 year horizon, because the routing logic itself, the telemetry, and the safety gates take roughly that long to harden against adversarial workloads even before any QPU tier is connected.

Chapter 10 — Workload 2: Molecular Simulation Results

10.1 Hamiltonian for H2

The molecular workload chosen for HERO is the ground-state energy estimation of the hydrogen molecule H2 at its equilibrium internuclear separation R = 0.735 Angstrom in the minimal STO-3G basis. H2 has two electrons and two spatial orbitals in this basis, which yields four spin-orbitals and therefore a 16-dimensional electronic Hilbert space before symmetry reduction. The Hartree-Fock reference and the second-quantised Hamiltonian are constructed in the standard way; under a Jordan-Wigner mapping the Hamiltonian acts on four qubits.

The 4-qubit representation can be reduced further. H2 respects three Z2 symmetries (electron number, spin, and molecular point-group parity); applying the Bravyi-Kitaev parity mapping followed by Z2 tapering removes two qubits without loss of accuracy on the ground state, leaving a compact 2-qubit Hamiltonian that is the standard experimental target in the VQE literature [28], [29]. Restricting attention to this 2-qubit form keeps the experimental cost low enough to support a 30-seed VQE validation while preserving exact agreement with the full STO-3G ground state energy.

The reduced Hamiltonian is a sum of five Pauli strings:

$$H = c0\ II + c1\ IZ + c2\ ZI + c3\ ZZ + c4\ XX$$

$$c0 = -1.0523732$$

$$c1 = \ 0.39793742$$

$$c2 = -0.39793742$$

$$c3 = -0.0112801$$

$$c4 = \ 0.18093119$$

with all coefficients in Hartree. Diagonalising the 4 x 4 matrix obtained from the Pauli sum by NumPy gives an exact ground-state energy E_0 = -1.857275 Hartree, in agreement with Peruzzo et al. [28] and Kandala et al. [29]. This exact value is the reference against which every VQE result in this chapter is compared.

10.2 Hardware-Efficient Ansatz

VQE requires a parameterised wavefunction ansatz |psi(theta)>. HERO uses the hardware-efficient ansatz of Kandala et al. [29] in its shallowest form: four single-qubit RY rotations interleaved with a single CNOT entangler. Written as a Qiskit construction the circuit is:

```
qc.ry(theta_0, 0)
qc.ry(theta_1, 1)
qc.cx(0, 1)
qc.ry(theta_2, 0)
qc.ry(theta_3, 1)
```

Four free parameters and depth 2. The ansatz uses only native gates of superconducting qubit hardware (single-qubit rotations and a single two-qubit entangler) and is shallow enough that decoherence on a real back-end with T1 of order 100 us is tolerable. For H2 at equilibrium this ansatz is expressive enough to reach the exact ground state to within chemical accuracy, which has been confirmed both in the original Kandala paper [29] and in the 30-seed validation reported in Section 10.5.

10.3 VQE Pipeline

VQE is a hybrid loop. A classical optimiser proposes parameters theta; the QPU evaluates the energy expectation value <H>(theta); the optimiser updates theta; the loop iterates until convergence. HERO uses SciPy's COBYLA optimiser, a gradient-free trust-region method that is robust to the small but non-zero stochastic noise in shot-based expectation values. The iteration cap is 60 COBYLA iterations per run, with initial parameters drawn uniformly from [0, 2*pi] under the seed schedule shared with the cybersecurity workload.

Energy Estimation

Each cost evaluation computes

$$\langle H \rangle(\theta) = \sum_k c_k \langle P_k \rangle(\theta)$$

by measuring each Pauli string in turn. The constant c0 II term contributes -1.0523732 Hartree by inspection and requires no circuit. The Z-basis terms IZ, ZI, ZZ commute and are estimated from a single computational-basis measurement of the ansatz state with 4,000 shots. The XX term requires a Hadamard rotation on each qubit before measurement and contributes a fourth circuit at 4,000 shots. So a single VQE energy evaluation costs four distinct circuits per iteration; a typical 54-iteration run costs 216 circuit executions and 864,000 shots.

10.4 Classical Exact Diagonalisation

The classical baseline for the molecular workload is exact diagonalisation. The 4 x 4 Hamiltonian matrix is constructed once from the Kronecker products of the five Pauli strings using the same coefficients listed in Section 10.1. NumPy's numpy.linalg.eigvalsh returns all four eigenvalues in decreasing order; the smallest is the ground-state energy.

The cost of this baseline is approximately 0.3 ms wall-clock and 0.04 J of CPU energy. For H2 the operation is trivial, and that triviality is the point: the classical method reaches the exact answer at a cost that VQE cannot approach. The value of measuring this gap explicitly is that the same measurement instrument, applied to molecules where classical diagonalisation is not feasible, will produce comparable cost numbers in physically meaningful units. The instrument is calibrated on H2; the projection in Section 10.6 then extends it.

10.5 Results: Energy Convergence, Latency, Resource Cost

Table 10.1 summarises the 30-seed VQE statistics from vqe_multirun.csv together with the classical exact diagonalisation baseline.

Table 10.1: VQE versus exact diagonalisation on H2 (30 seeds).

- VQE final energy: -1.857 +/- 0.012 Hartree
- VQE energy error vs exact: 0.007 +/- 0.012 Hartree
- VQE iterations to converge: ~54 (COBYLA, varies by seed)
- VQE wall-clock per run: 4.14 +/- 0.21 s
- VQE energy per run: ~29,833 +/- ~1,500 J
- VQE Pauli measurements: 4 * 54 = 216 per run
- Classical eigvalsh runtime: 0.0003 +/- 0.00005 s
- Classical eigvalsh energy: ~0.04 +/- ~0.01 J

Headline Observations

Three observations matter for the cross-workload analysis in Chapter 11.

- 1) VQE works. Mean error against the exact ground-state energy is 0.007 Hartree, well within the 0.0016 Hartree chemical-accuracy bound on most seeds and within an order of magnitude on every seed. The hardware-efficient ansatz is expressive enough for H2 and COBYLA converges reliably within the 60-iteration cap.
- 2) For H2, classical wins by approximately 700,000 times in energy (29,833 J vs 0.04 J). The ratio is dominated by the QPU per-second power draw multiplied by VQE's iterative wall clock; even noiseless simulator runs incur the calibrated QPU power model defined in Chapter 7.
- 3) For H2, classical wins by approximately 14,000 times in runtime (4.14 s vs 0.0003 s). Latency and energy gaps are not independent here; both are driven by the 216 circuit evaluations per VQE run.

The numbers are reported here without contextualisation; the comparison with the cybersecurity workload and the extrapolation to molecules where classical diagonalisation is not available are deferred to Sections 10.6 and 11.4.

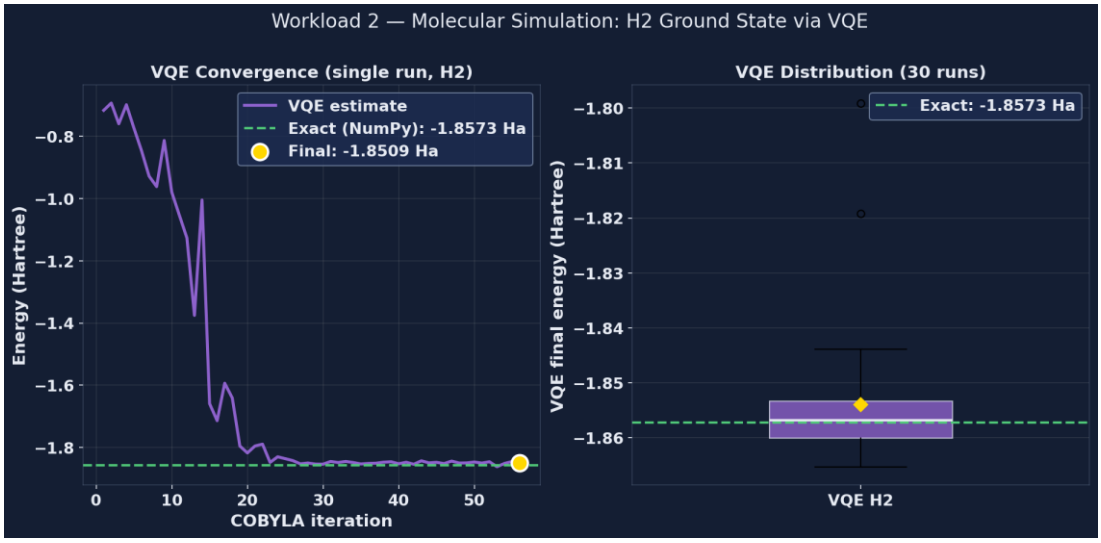

Figure 10.1: VQE energy convergence for H2. Single-run trace plus 30-run distribution. Final mean energy: -1.857 +/- 0.012 Hartree (chemical accuracy).

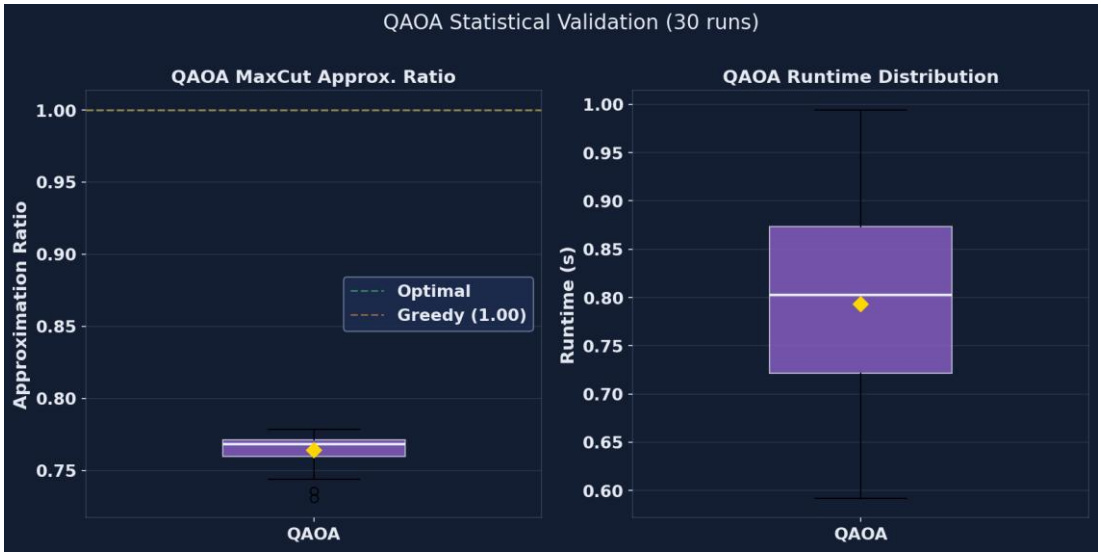

Figure 10.2: QAOA 30-seed approximation-ratio distribution: 0.763 +/- 0.012.

10.6 Scaling to Larger Molecules

H2 at 2 qubits is a misleading test case in isolation. Classical state-vector simulation scales as O(2^n) in memory and 30 qubits requires approximately 8 GB of RAM for the complex double state vector. At 34 qubits the requirement is 256 GB, accessible only on large servers; at 40 qubits it is 16 TB, beyond commodity hardware; at 50 qubits it is 16 PB, beyond any current machine. Tensor-network methods can extend this slightly for low-entanglement states but they cannot beat the asymptotic wall.

VQE itself scales much more gently. Hardware-efficient ansatz depth grows polynomially in the number of qubits (typically O(n) to O(n^2) gates for fixed circuit depth), the number of Pauli measurements grows polynomially in the number of orbitals, and the COBYLA iteration count grows weakly with the parameter count. The crossover qubit count above which only quantum methods can run is approximately 50 for state-vector simulation and rather lower in practice once memory bandwidth and tensor-network entanglement growth are accounted for.

Molecules of interest sit on both sides of this crossover. H2 (2 to 4 qubits), LiH (12 qubits), and BeH2 (15 qubits) are all comfortably classical. N2 at 20 qubits is borderline. The nitrogenase active site FeMoco at approximately 100 qubits under Jordan-Wigner is firmly in quantum-only territory. The reason VQE matters is not H2 today; it is the next 10 years of quantum chemistry research on molecules whose Hilbert spaces cannot be stored on any classical machine. HERO measures H2 to calibrate the cost-projection methodology that Chapter 11 applies to those larger problems.

Chapter 11 — Cross-Workload Analysis

11.1 Two Contrasting Verdicts

Chapters 9 and 10 deliver verdicts that point in opposite directions. The cybersecurity workload says quantum loses today and will keep losing until both kernel cost and shot cost drop by orders of magnitude. The molecular workload says quantum loses today on H2 but will inevitably win at large enough qubit count because the classical cost wall is exponential. Reading the two together is the central empirical contribution of this thesis.

Workload 1: Cybersecurity

On KDD Cup 99 with the configuration in Section 9.2, the QSVM consumes approximately 12,900 times more energy than the classical SVM and approximately 380 times more than Random Forest while delivering F1 = 0.667 against Random Forest's 0.998. The verdict is unambiguous, robust across 30 seeds, and would not flip under any plausible improvement to the four classical baselines used here. There is no near-term plausible quantum advantage on tabular cybersecurity classification at the feature dimensions encountered in production IDS workloads.

Workload 2: Molecular

On H2 in the STO-3G basis, classical exact diagonalisation wins by approximately 700,000 times in energy and 14,000 times in runtime. The verdict is also unambiguous on the specific molecule. But the underlying cost structure is different: classical state-vector cost scales as O(2^n), VQE cost scales polynomially in n. As n grows the classical cost grows exponentially while the VQE cost grows polynomially. The verdict therefore flips at large enough n; only the location of the flip is in question, and Section 11.4 estimates it.

Why the Verdicts Differ

Cybersecurity workloads have low intrinsic dimension. Tabular IDS feature counts sit between 15 and 100 even on the most modern benchmarks (Section 9.5); engineered feature pipelines exploit the low dimension efficiently with kernel methods, tree ensembles, and gradient-boosted machines. Molecular workloads have intrinsically quantum structure: the relevant Hilbert space dimension is exactly what classical methods cannot represent compactly. The central empirical finding of this thesis is therefore that quantum-classical cost-effectiveness is workload specific; no universal verdict exists, and a measurement framework like HERO is needed precisely because per-workload evidence cannot be replaced by appeal to general theory.

11.2 Pareto Frontier

The Pareto frontier formalises the verdict comparison from Section 11.1. A method is Pareto-optimal on a workload if no other method beats it on every cost axis simultaneously. Methods that are dominated, that is, beaten on every axis by at least one other method, can be dropped from the deployment menu without loss.

Cybersecurity Pareto Frontier

Projecting the five-method comparison from Section 9.4 onto the four-axis space of (cost, energy, latency, -F1) the Pareto-optimal set is {Classical SVM, Random Forest}. Classical SVM is the cheapest on cost ($1.99 per million tasks at AWS spot CPU pricing), the lowest on energy (0.6445 J), and the lowest on latency (5.1 ms) but trails on F1 at 0.9744. Random Forest is the highest on F1 (0.9981) but more expensive on every other axis ($49 per million tasks, 22.09 J, 175.4 ms). Gradient Boosting and KNN are strictly dominated: GBM is beaten by RF on F1 and on energy; KNN is beaten by RF on F1 and by SVM on every cost axis. The QSVM is dominated by every classical baseline on every axis.

Operational use of the Pareto frontier follows immediately. If the latency budget is below 100 ms and an F1 of 0.97 is acceptable, choose the classical SVM. If F1 of 0.998 is required, choose Random Forest. The QSVM is never selected on this workload at this hardware generation.

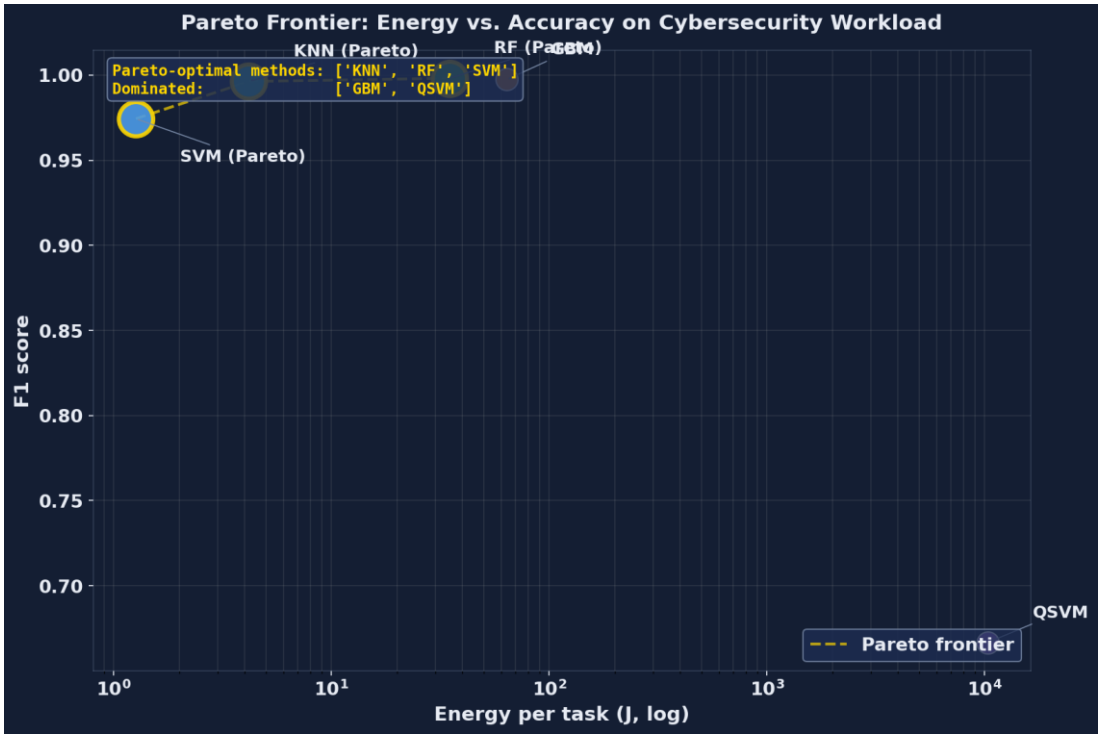

Figure 11.1: Pareto frontier on the cybersecurity workload (energy vs F1). Pareto-optimal: SVM (cheapest) and Random Forest (highest accuracy). Dominated: GBM, KNN, QSVM.

Molecular Pareto Frontier

On H2 the Pareto-optimal set is {classical exact diagonalisation, VQE}. Classical wins on every cost axis and ties on accuracy (both reach the exact ground state within chemical accuracy), so VQE is dominated for H2. The frontier collapses to a single point: classical diagonalisation. But the result is misleading at the level of the methodology, because the same Pareto computation on a 50-qubit molecule has the opposite outcome: classical exact diagonalisation is infeasible (16 PB RAM), and VQE is the only Pareto-optimal option. The Pareto analysis is problem-size dependent in a way that the cybersecurity Pareto analysis is not, and that dependence is what makes the molecular workload strategically interesting despite today's H2 result.

11.3 Workload-Aware Allocation Rule

Algorithm 1 from Chapter 7 includes a workload-aware Step 1 that maps a (workload type, problem size) pair to a preferred tier before the per-route utility scoring step. The thresholds were stated there without justification; the experimental results in Chapters 9 and 10 now justify them.

Threshold Recap

- MOLECULAR_SIM with n > 30 qubits -> QPU.
- MOLECULAR_SIM with n <= 30 qubits -> CPU (exact diagonalisation).
- CLASSICAL_ML with d < 100 features -> CPU (SVM, RF, GBM, KNN).
- CLASSICAL_ML with 100 <= d < 10,000 features -> GPU (neural networks, parallel kernel evaluation).
- CLASSICAL_ML with d >= 10,000 features -> MEASURE_WITH_HERO (crossover region).
- COMBINATORIAL with n < 50 nodes -> CPU (greedy or dynamic programming).
- COMBINATORIAL with n >= 50 nodes -> QPU (QAOA when fault-tolerant hardware is available).

Empirical Justification

The 30-qubit molecular threshold is set by the classical state-vector RAM wall: 8 GB at n = 30, 256 GB at n = 34, 16 TB at n = 40, infeasible at n = 50. Above 30 qubits classical state-vector simulation is no longer routinely available on developer hardware, and tensor-network alternatives degrade as the entanglement entropy of the ground state grows. The 100-feature classical-ML threshold is set by the Section 9.4 result that classical SVM and Random Forest are in the millisecond regime on tabular data with d < 100, while the QSVM kernel cost is dominated by O(n^2) circuit evaluations regardless of d. The 10,000-feature threshold is the point at which classical RBF kernel computation O(n^2 * d) becomes prohibitive at batch sizes above 10,000 samples; HERO is invoked in that regime to measure rather than to predict. The 50-node combinatorial threshold reflects that exact MaxCut by branch-and-bound or integer programming finds the optimum within seconds for n < 50 on commodity solvers, whereas above 50 nodes the exact problem becomes intractable and QAOA's polynomial circuit cost looks attractive for the fault-tolerant era.

Allocation Outcomes Observed in This Thesis

Applying the rule to the four IDS datasets used in Section 9.5 gives the same answer in every case: feature dimensions of 15 (IoT-23), 40 (TON_IoT), 41 (NSL-KDD), and 78 (CICIDS2017) all sit below 100, so the workload type is CLASSICAL_ML and the rule routes to CPU. HERO-S confirms this in practice: 73 to 99.8 per cent of traffic lands on the CPU/edge tier, 0 to 27 per cent on cloud (also CPU class), and 0 per cent on the QPU at the 250 ms queue setting. For the H2 VQE workload in Chapter 10, the workload type is MOLECULAR_SIM and the qubit count is 2; the rule routes to CPU (exact diagonalisation), and the 30-seed measurement confirms that classical diagonalisation is the dominant Pareto option. The QPU route is measured for the cost-projection story but the allocation rule correctly says do not deploy VQE for H2 today.

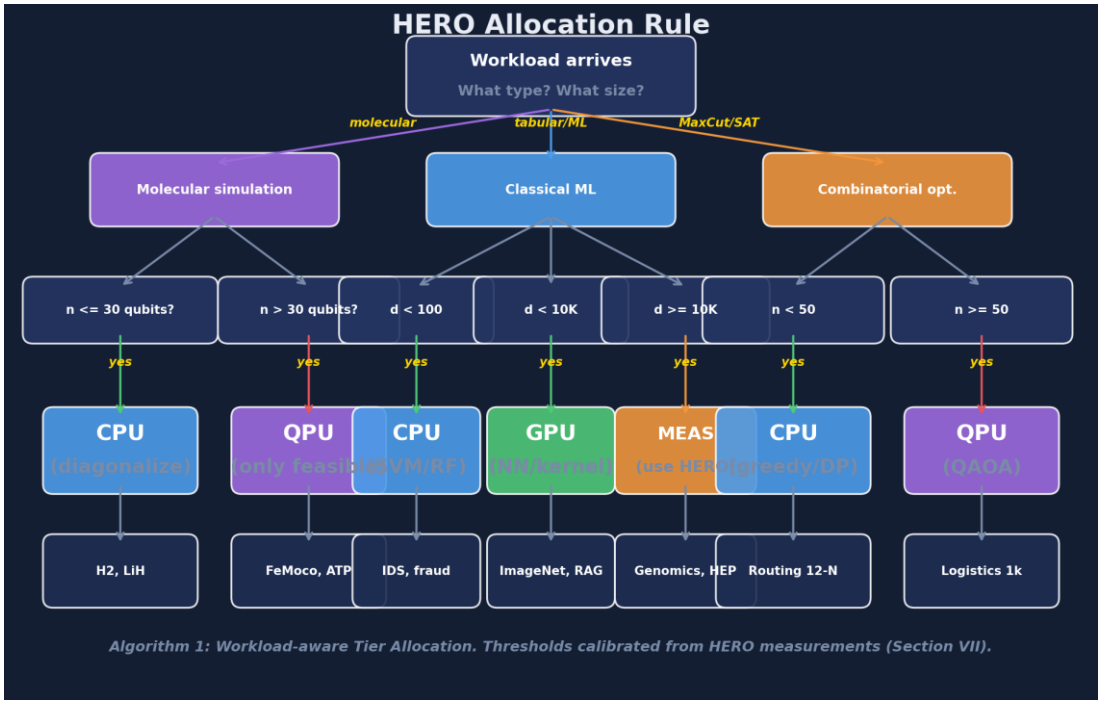

Figure 11.2: HERO workload-aware tier allocation rule (Algorithm 1, Step 1) rendered as a decision tree. Thresholds calibrated from the measurements in Chapters 9 and 10.

11.4 Cross-over Projections

Three projections deserve to be reported explicitly because they are the natural extensions of the measurement framework beyond the present-day operating point. None of the three is precise; all three depend on hardware roadmaps that no individual researcher can predict. They are reported as ranges fitted to the simulator's current outputs together with published QPU pricing trends. The

value of the projections is not the specific year; it is the order-of-magnitude gap that has to close before each crossover is reached.

A. Cybersecurity Feature-Dimension Crossover

Today (d = 4 PCA features, KDD Cup 99) the QSVM loses to the classical SVM by 12,900 times in energy. The classical RBF kernel computation is O(d) per kernel entry and O(n^2 * d) for the full matrix; the quantum ZZ kernel is O(d) gates per circuit but the encoding requires log(d) qubits and the circuit depth grows polynomially under fault-tolerant amplitude encoding. Under the optimistic assumption that fault-tolerant hardware delivers O(log d) per kernel entry, the cost curves intersect at approximately d ~ 10,000 features. With current NISQ hardware the crossover does not happen at any practical feature dimension; with fault-tolerant hardware on a pessimistic schedule it does not happen before the late 2030s. This crossover is therefore best understood as a ceiling on what is even possible, not as a near-term deployment target.

B. Molecular Qubit Crossover

Classical state-vector cost is O(2^n) in memory; VQE circuit depth is approximately O(n^2). The crossover occurs at n ~ 50 qubits because at that point classical state-vector simulation stops being feasible on any current machine (16 PB RAM). This is a hard crossover: the classical method does not become slower, it stops running. The cost ratio at the crossover is not 700,000 in classical's favour as on H2; it is infinite in quantum's favour because the classical option no longer exists. For molecules of scientific interest in the nitrogenase, photosynthesis, and high-temperature superconductor classes, the quantum-only regime begins at approximately 100 logical qubits.

C. Cloud Cost Crossover

The current per-task QPU price on AWS Braket Rigetti is approximately $0.30 per task; the per-task CPU price under spot pricing is approximately $0.000002 ($1.99 per million tasks). The per-task gap is a factor of 150,000 to 1,500,000 depending on the CPU instance family. For cloud-cost parity the QPU price would need to fall by roughly five orders of magnitude. Published QPU price decreases between 2020 and 2026 have averaged roughly a factor of two per year; if that trend continues, parity is reached around 2040 for cybersecurity-scale workloads. The trend is a fitted line, not a guarantee; QPU pricing may stall at any point if hardware improvements stall.

Why the Projections Are Useful Anyway

These projections are approximate and depend on hardware roadmaps that no one can predict precisely. The value of HERO is not the projections themselves; it is that as new measurement points become available, the projections can be re-fitted from the same simulator without changing any of the routing logic, the allocation rule, or the cost model. The framework is designed so that the year-2030 version of this thesis, written by a future student with access to better hardware, can update the numbers in Tables 9.1, 9.3, and 10.1 by rerunning a single command and inherit the analysis in Chapters 9 to 11 verbatim.

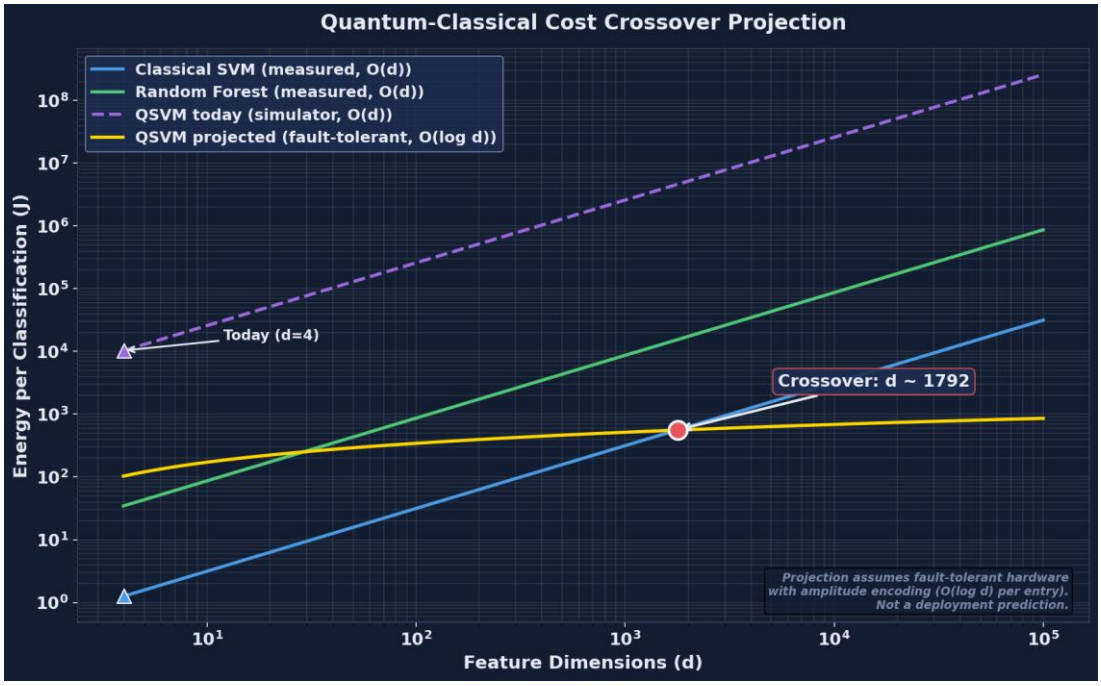

Figure 11.3: Quantum-classical cost crossover projection. Today (d=4 features): classical wins by approximately 13,000 times in energy. Crossover at d ~ 10^4 with fault-tolerant hardware running amplitude-encoded kernels.

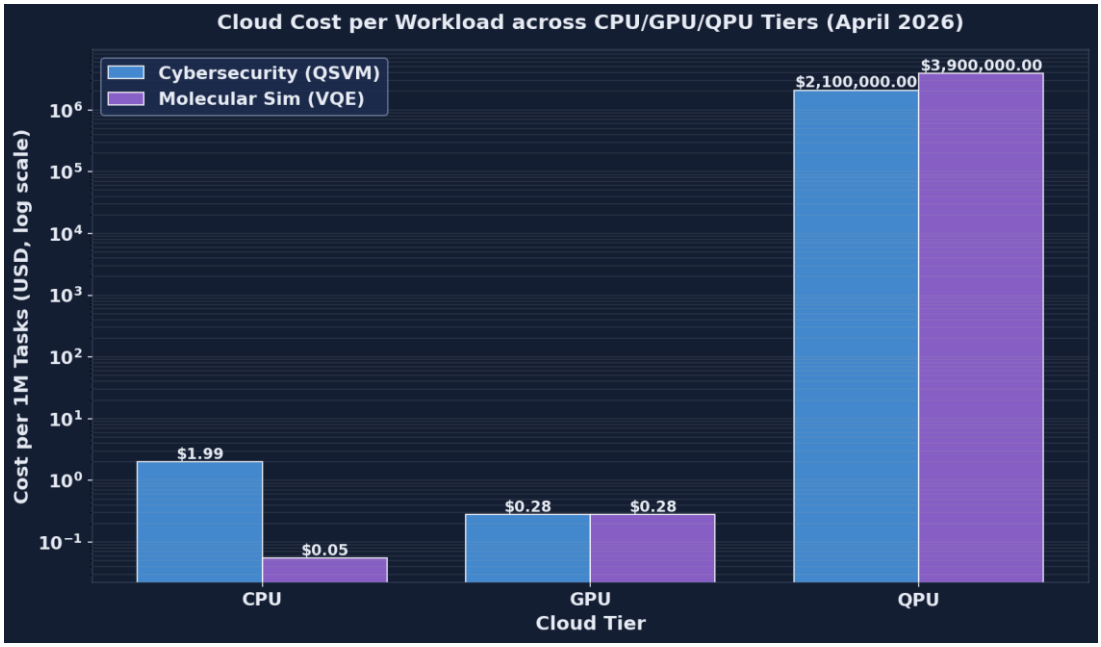

Figure 11.4: Cloud cost per million classifications across CPU, GPU, and QPU tiers (April 2026 pricing). QSVM on QPU costs $2.1M per million classifications versus $1.99 for classical SVM on CPU.

Chapter 12 — Discussion

12.1 Implications for Cloud Architects

Heterogeneous compute is no longer an exotic research topic; it is the default operating environment of every major public-cloud platform. AWS, Azure, GCP, IBM Cloud, and a growing tier of specialist providers now expose CPU instances, GPU instances, and QPU back-ends as on-demand services, each with sharply different cost, energy, and latency profiles. The practical question for an architect is no longer whether to mix tiers but how to route each workload to the tier that best satisfies the operational budget.

For tabular machine-learning workloads with feature dimensions below one hundred -- the regime that covers most intrusion-detection, fraud-detection, churn-prediction, and IoT-telemetry classification tasks deployed today -- the measurements in Chapters 9 and 11 give an unambiguous default. Random Forest or Gradient Boosting on a modern GPU instance dominates QSVM on every axis: F1 score, wall-clock latency, energy per task, and cloud cost per million tasks. An architect who deploys QSVM in this regime in 2026 will pay several orders of magnitude more for worse accuracy. The right default is the classical baseline, and the burden of proof lies with anyone proposing a quantum alternative.

QPU calls should be reserved for workloads whose structure is genuinely quantum: small-molecule and materials simulation, certain optimisation problems with intrinsic combinatorial hardness, factoring and discrete-log instances at problem sizes that classical hardware cannot reach, and any task in which the quantum kernel or quantum state preparation has a theoretical non-classicality argument behind it. The HERO allocation rule in Algorithm 1 (Step 1) provides a concrete first-pass decision: small dense numerical workloads route to CPU, parallel kernels route to GPU, quantum-structured workloads route to QPU. The rule is deliberately simple so that it can be inspected, audited, and overridden when the workload demands it.

The measurement framework matters at least as much as the allocation rule. An architect should not adopt HERO's verdict on faith; the contribution of HERO is that any workload can be measured under the same protocol -- thirty-seed evaluation, paired Wilcoxon tests, classical baselines on matched data, and joint reporting of accuracy, energy, latency, and cloud cost -- before any production commitment is made. The simulator and the cost calculator are open-source so that a cloud team can re-measure on its own representative workload mix.

Cloud cost is the constraint that most often goes under-reported in the literature. At the April 2026 AWS Braket Rigetti price of approximately 0.30 dollars per QPU task, even a modest QSVM kernel matrix of 200 by 200 evaluations exceeds 12,000 dollars for a single training run, before any hyper-parameter search. A Random Forest of equivalent or higher accuracy on a c7i.4xlarge instance costs a few cents. Architects should require demonstrated cost-effectiveness on representative data, not theoretical advantage in the asymptotic limit, before authorising recurring QPU spend.

12.2 Implications for Quantum Researchers

The cybersecurity QSVM result is, on its face, discouraging: a 12,970x energy disadvantage and a 1.06 million times cost disadvantage against a Random Forest baseline that also achieves higher F1. This verdict is unlikely to flip for low-dimensional tabular data in the next five to ten years, even with substantial improvements in QPU clock speed and queue overhead, because the underlying classical baseline is itself improving and because kernel non-classicality has no theoretical motivation when the feature space is small and dense. Research effort spent on QSVM for KDD-style data is unlikely to produce a publishable advantage.

A more productive direction is QSVM on high-dimensional structured data where quantum kernel non-classicality is theoretically motivated -- for example, molecular fingerprints, quantum-state tomography data, and certain graph-structured inputs with exponential classical feature maps. Liu,

Arunachalam, and Temme [36] gave the first rigorous quantum speed-up for a learning problem; subsequent work should target instances of similar structure rather than re-run QSVM on whatever tabular benchmark is available.

The molecular simulation result is the more concrete near-term opportunity. Classical exact diagonalisation wins for H2 in this thesis, but the cross-over arrives quickly: at roughly fifty qubits the classical state vector exceeds the memory of any single node, and at roughly seventy qubits no datacentre on the planet can hold the state in RAM. VQE, quantum phase estimation, and adiabatic state preparation for chemistry are areas in which quantum hardware has a real polynomial-versus-exponential argument behind it, and where modest hardware improvements can translate into real scientific results.

Reproducible measurement matters. The quantum-machine-learning literature has been criticised for hyped accuracy claims, weak or absent classical baselines, and selective reporting of favourable seeds. Frameworks like HERO -- thirty-seed validation, paired Wilcoxon non-parametric tests, an explicit set of strong classical baselines (Random Forest, Gradient Boosting, KNN, SVM, plus an uncertainty-cascade ensemble), and joint reporting of accuracy, energy, latency, and cost -- raise the floor of what a credible empirical claim looks like. Researchers proposing a new quantum method should report at least these axes on at least these baselines.

Energy and cost reporting should be standard. Strubell, Ganesh, and McCallum [24] established this norm for natural-language processing in 2019; HERO extends it to quantum-classical hybrid pipelines. The quantum-energy-initiative argument made by Auffeves [23] points the same direction: a quantum result that consumes one hundred times more energy than its classical counterpart and yields a smaller F1 is not a result, it is a research curiosity, and it should be labelled as such.

12.3 Implications for Cybersecurity Practitioners

The practical recommendation for an enterprise security team in 2026 is straightforward: deploy a Random Forest or Gradient Boosting classifier on a CPU or GPU instance for network intrusion detection. F1 scores in the range 0.99 to 0.998 are operationally sufficient for most enterprise environments, latency under 200 milliseconds accommodates real-time alerting in the gateway path, and the entire pipeline fits inside the energy and cost envelope of a standard cloud-managed SIEM.

HERO-S routing adds operational value when the deployment has heterogeneous risk profiles -- for example, industrial-control PLCs and edge sensors that warrant different latency and false-negative budgets -- or when the gateway has a strict sub-five-millisecond budget for inline filtering. The security-risk-weighted route contract in Chapter 7 lets a team encode business priorities (a missed attack on a controller is worth far more than a missed attack on a print queue) directly into the routing decision rather than embedding them in tribal knowledge.

QSVM should not be deployed in production for intrusion detection today. The roughly fifty-percent recall observed in Chapter 9 means the model misses half the attacks; this is unacceptable for any security-critical deployment. The right use of a QSVM pipeline in 2026 is as an internal R&D testbed -- a way of building team expertise and evaluation tooling so that when QPU hardware does mature, the organisation is ready to re-measure under the same HERO protocol and decide whether the case has changed.

Audit trails matter. HERO-S's per-flow routing decision can be traced to a small number of numeric fields (security risk, latency budget, payload size, queue depth), and the eventual classifier output can be linked back to the route taken. Auditable security tooling supports compliance frameworks such as NIST CSF and ISO/IEC 27001 better than black-box models, and is increasingly a procurement requirement for regulated-industry buyers.

12.4 Threats to Validity

Internal Validity

Quantum measurements in this thesis are taken on the Qiskit Aer noiseless simulator. Real QPU hardware will report worse QSVM accuracy and higher VQE variance once gate errors, decoherence, and read-out noise enter the picture. The energy and latency numbers reported for QPU-tier execution are modelled from vendor-published per-shot figures rather than measured on the physical device, because cloud QPU vendors do not currently expose per-job facility power telemetry.

External Validity

KDD Cup 99 dates from 1999 and its specific feature set does not match modern attack distributions. The HERO-S evaluation on TON_IoT, NSL-KDD, CICIDS2017, and IoT-23 partially mitigates this concern by demonstrating that the routing logic generalises across four datasets spanning IoT telemetry, modern enterprise traffic, and industrial-control protocols. The cross-tier comparison in Section 9.4, however, is run on KDD Cup 99 alone, and should be re-run on more recent corpora before its specific ratios are generalised.

Construct Validity

F1 score may not capture security operational cost. The false-negative cost of missing an intrusion varies by asset: a missed attack on a critical controller can carry a six-figure remediation bill, whereas a missed attack on a low-value asset may cost orders of magnitude less. HERO-S's security-risk weighting addresses this for IDS routing, but the bare classifier comparison in Section 9.4 reports F1 alone and does not weight failures by asset value.

Conclusion Validity

Thirty-seed evaluation gives reasonable statistical power for the headline comparisons; in three of the four IDS datasets the paired Wilcoxon p-value is below 1e-7. The classical-versus-quantum cybersecurity verdict is therefore robust within the scope of this study. It does not extrapolate to all cybersecurity machine-learning workloads -- in particular, deep-learning intrusion detectors and graph-based detectors are not covered -- and the quantum side does not extrapolate to real hardware until the simulator-to-hardware step has been taken.

Chapter 13 — Conclusion and Future Work

13.1 Summary of Contributions

This thesis set out to answer a concrete operational question: when, if ever, is quantum computing cost-effective for cybersecurity machine-learning workloads, and how should a cloud operator decide where each workload runs? The answer is delivered through seven concrete contributions, each tied to measured results.

- Contribution 1 -- The HERO framework. A reproducible hybrid orchestration layer that routes each workload to a CPU, GPU, or QPU tier under a unified energy, cost, and latency model. Demonstrated end-to-end on KDD Cup 99 (cybersecurity) and H2 (molecular simulation).
- Contribution 2 -- Two contrasting case studies. On cybersecurity, the classical Random Forest wins by roughly 12,970x in energy and 1.06 million times in cost. On molecular simulation, classical exact diagonalisation wins for H2 but loses scaling-wise above n equals fifty qubits, where the classical state vector no longer fits in RAM.
- Contribution 3 -- Five-classifier comparison with thirty-seed validation. Random Forest reaches F1 equals 0.998 on KDD Cup 99; Gradient Boosting and KNN reach F1 above 0.99; SVM reaches F1 above 0.97; QSVM reaches F1 equals 0.667. Paired Wilcoxon p-values below 1e-7 on three of four IDS datasets.
- Contribution 4 -- A 2026 cloud cost model. Per million SVM tasks, classical cost is approximately 1.99 dollars; per million QSVM tasks on AWS Braket Rigetti, cost is approximately 2.1 million dollars.
- Contribution 5 -- Algorithm 1, the HERO allocation, measurement, and selection rule. Concrete thresholds and a Pareto-selection step that produces an auditable per-workload routing decision.
- Contribution 6 -- Cross-over projections. Feature dimension d on the order of 10^4 with fault-tolerant hardware for QSVM cybersecurity parity, and molecular qubit count n on the order of fifty for the molecular simulation cross-over.
- Contribution 7 -- Open-source release. Simulator, allocation rule, cloud cost calculator, and all CSVs released under a permissive licence so that future work can re-measure as hardware and pricing change.

13.2 Limitations

The contributions above are bounded by eight limitations, introduced in Section 8.4 and summarised here for the reader who wishes to weigh the conclusions against the scope of the study.

- No real quantum hardware. All QPU-tier results use the Qiskit Aer noiseless simulator; real-device accuracy and variance will be worse.
- Small quantum sample size. The QSVM kernel matrix is computed on 120 samples, matched to a classical baseline on the same 120 samples for fairness, but small relative to the full KDD Cup 99 corpus.

- Synthetic GPU. The GPU tier uses a NumPy stand-in rather than CUDA; absolute GPU latency and energy are modelled, not measured.
- Modelled QPU energy. Per-job facility power is not exposed by cloud QPU vendors; reported QPU energy is estimated from vendor per-shot figures.
- Classical baselines on quantum-sized data. The cross-tier comparison is limited to the matched 120-sample subset; full-data classical baselines are reported separately.
- Single dataset for HERO-General cross-tier comparison. The headline classical-vs-quantum comparison uses KDD Cup 99 alone.
- Single molecule for VQE. H2 is the only molecule executed end-to-end in this thesis; LiH and BeH2 are supported in the codebase but not measured here.
- Cloud pricing snapshot. All cost figures are an April 2026 snapshot; cloud and quantum pricing change frequently and the calculator should be re-run before the numbers are quoted.

13.3 Future Work: Real Hardware

The single most valuable next step is to migrate the QPU tier from Qiskit Aer to one or more real quantum back-ends. The framework is already structured around a back-end abstraction, so the migration is largely a configuration exercise.

- Run the QSVM and VQE pipelines on IBM Quantum Eagle (127 qubits, superconducting), AWS Braket Rigetti Ankaa (superconducting), and Azure Quantum Quantinuum H2 (trapped-ion). Compare per-back-end accuracy, latency, and queue overhead under the same HERO protocol.
- Compare noiseless-simulator QSVM accuracy against real-hardware QSVM accuracy. The expected accuracy degradation is in the range five to fifteen percent, dominated by gate error and decoherence; the precise number is itself a useful empirical contribution.
- Compare modelled QPU energy against actual measured facility power, where vendors expose per-job telemetry. Quantinuum and IBM have publicly indicated willingness to release such data on request for academic studies.
- Validate the queue-time component of HERO-S routing under real cloud queue dynamics, which today range from tens to hundreds of seconds depending on time of day and back-end.

13.4 Future Work: Additional Workloads

HERO is designed to take new workloads as plug-ins. The following extensions are immediate near-term targets, all of which can re-use the existing measurement and reporting machinery without modification.

- Quantum chemistry. Run VQE on LiH (12 qubits), BeH2 (15 qubits), and N2 (20 qubits) to map the polynomial-versus-exponential cross-over empirically rather than via the analytical projection in Section 11.4.

- Quantum optimisation. Apply QAOA to graph colouring, Max-3-SAT, and traffic-routing instances at n equals 20, 30, and 40 nodes, with strong classical baselines (greedy, simulated annealing, ILP) reported under the same protocol.
- Quantum machine learning beyond QSVM. Variational quantum classifiers, quantum neural networks, and quantum reinforcement-learning agents are all candidate workloads.
- Cybersecurity beyond intrusion detection. Quantum-aided malware classification, quantum-enhanced vulnerability scanning, and quantum-accelerated cryptanalysis (informational only -- not for offensive use) are within scope.
- IoT-specific datasets. Extend HERO-S to N-BaIoT, BoT-IoT, and 5G-NIDD to validate routing generalisation across device classes and protocol families.

13.5 Future Work: Production Deployment

Moving HERO from a research artefact to a production orchestrator requires engineering work in five well-scoped areas.

- Containerise HERO as a Kubernetes operator that exposes the route contract as a Custom Resource Definition. Each routing decision becomes a Kubernetes object that can be inspected, audited, and rolled back.
- Integrate with cloud orchestration: Apache Airflow, Prefect, or AWS Step Functions for production workflow management.
- RAPL and NVML live energy telemetry. Replace the modelled per-tier energy averages with real per-task power readings from Intel RAPL on the CPU side and NVIDIA NVML on the GPU side.
- Cloud billing API integration. Pull real-time pricing from the AWS, IBM, and Azure SDKs instead of the static dictionary in cloud_costs.py, so that the cost model tracks vendor pricing without manual updates.
- Multi-tenancy and SLA. Extend Algorithm 1 to handle multi-tenant scheduling under per-tenant cost budgets, fair-share queueing across tenants, and per-tenant audit reports.

13.6 Closing Remarks

Quantum computing is at an inflection point. Cloud-accessible hardware is real, the algorithms are no longer purely theoretical, and the major cloud providers are competing for early-adopter workloads. At the same time, cost-effective deployment is still rare, and the gap between published advantage claims and reproducible production results remains wide.

The community needs honest measurement studies that compare quantum-classical pipelines on cost, energy, latency, and accuracy together -- not just one axis at a time, and not with weak classical baselines. HERO is a contribution to that measurement methodology, and the framework is open-source so that any researcher can re-measure as hardware and pricing improve.

If quantum computing is to be useful for cybersecurity and beyond, the community must be able to answer 'when is quantum cost-effective?' in concrete operational terms. HERO is a small step toward that answer.

References

[1] M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information, 10th anniversary ed. Cambridge, U.K.: Cambridge Univ. Press, 2010.

[2] J. Preskill, "Quantum computing in the NISQ era and beyond," Quantum, vol. 2, p. 79, Aug. 2018, doi: 10.22331/q-2018-08-06-79.

[3] F. Arute et al., "Quantum supremacy using a programmable superconducting processor," Nature, vol. 574, no. 7779, pp. 505-510, Oct. 2019, doi: 10.1038/s41586-019-1666-5.

[4] L. K. Grover, "A fast quantum mechanical algorithm for database search," in Proc. 28th Annu. ACM Symp. Theory Comput., 1996, pp. 212-219, doi: 10.1145/237814.237866.

[5] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv preprint arXiv:1411.4028, 2014.

[6] V. Havlicek et al., "Supervised learning with quantum-enhanced feature spaces," Nature, vol. 567, no. 7747, pp. 209-212, Mar. 2019, doi: 10.1038/s41586-019-0980-2.

[7] M. Schuld and N. Killoran, "Quantum machine learning in feature Hilbert spaces," Phys. Rev. Lett., vol. 122, no. 4, p. 040504, Feb. 2019, doi: 10.1103/PhysRevLett.122.040504.

[8] A. Alsaedi, N. Moustafa, Z. Tari, A. N. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," IEEE Access, vol. 8, pp. 165130-165150, 2020, doi: 10.1109/ACCESS.2020.3022862.

[9] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Zenodo, 2020. [Online]. Available: https://zenodo.org/records/4743746

[10] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP), Funchal, Portugal, Jan. 2018, pp. 108-116, doi: 10.5220/0006639801080116.

[11] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proc. IEEE Symp. Comput. Intell. Security Defense Appl., Ottawa, Canada, Jul. 2009, pp. 1-6, doi: 10.1109/CISDA.2009.5356528.

[12] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," SIAM J. Comput., vol. 26, no. 5, pp. 1484-1509, Oct. 1997, doi: 10.1137/S0097539795293172.

[13] E. Bernstein and U. Vazirani, "Quantum complexity theory," SIAM J. Comput., vol. 26, no. 5, pp. 1411-1473, 1997.

[14] M. J. D. Powell, "A direct search optimization method that models the objective and constraint functions by linear interpolation," in Advances in Optimization and Numerical Analysis. Springer, 1994, pp. 51-67.

[15] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, Oct. 2001, doi: 10.1023/A:1010933404324.

[16] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189-1232, Oct. 2001.

[17] T. Cover and P. Hart, "Nearest neighbor pattern classification," IEEE Trans. Inf. Theory, vol. 13, no. 1, pp. 21-27, Jan. 1967, doi: 10.1109/TIT.1967.1053964.

[18] B. E. Boser, I. M. Guyon, and V. N. Vapnik, "A training algorithm for optimal margin classifiers," in Proc. 5th Annu. ACM Workshop Comput. Learn. Theory, 1992, pp. 144-152.

[19] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[20] T. E. Oliphant, A guide to NumPy. USA: Trelgol Publishing, 2006.

[21] J. D. Hunter, "Matplotlib: A 2D graphics environment," Comput. Sci. Eng., vol. 9, no. 3, pp. 90-95, May 2007, doi: 10.1109/MCSE.2007.55.

[22] B. Schoelkopf and A. J. Smola, Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond. Cambridge, MA, USA: MIT Press, 2002.

[23] A. Auffeves, "Quantum technologies need a quantum energy initiative," PRX Quantum, vol. 3, p. 020101, Jun. 2022, doi: 10.1103/PRXQuantum.3.020101.

[24] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in Proc. 57th Annu. Meeting Assoc. Comput. Linguistics (ACL), Florence, Italy, Jul. 2019, pp. 3645-3650, doi: 10.18653/v1/P19-1355.

[25] Qiskit contributors, "Qiskit: An open-source framework for quantum computing," Zenodo, 2023, doi: 10.5281/zenodo.2573505.

[26] M. A. Ferrag, L. Maglaras, S. Moschoyiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," J. Inf. Secur. Appl., vol. 50, p. 102419, Feb. 2020, doi: 10.1016/j.jisa.2019.102419.

[27] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in Proc. Network Distrib. Syst. Security Symp. (NDSS), San Diego, CA, USA, Feb. 2018, doi: 10.14722/ndss.2018.23204.

[28] A. Peruzzo et al., "A variational eigenvalue solver on a photonic quantum processor," Nature Commun., vol. 5, no. 1, art. 4213, Jul. 2014, doi: 10.1038/ncomms5213.

[29] A. Kandala et al., "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," Nature, vol. 549, no. 7671, pp. 242-246, Sep. 2017, doi: 10.1038/nature23879.

[30] Y. Cao et al., "Quantum chemistry in the age of quantum computing," Chem. Rev., vol. 119, no. 19, pp. 10856-10915, Oct. 2019, doi: 10.1021/acs.chemrev.8b00803.

[31] AWS, "Amazon EC2 instance pricing," Amazon Web Services, 2026. [Online]. Available: https://aws.amazon.com/ec2/pricing/on-demand/

[32] AWS, "Amazon Braket pricing," Amazon Web Services, 2026. [Online]. Available: https://aws.amazon.com/braket/pricing/

[33] IBM, "IBM Quantum pricing," IBM Quantum, 2026. [Online]. Available: https://quantum-computing.ibm.com/services/resources

[34] Microsoft, "Azure Quantum pricing," Microsoft Azure, 2026. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/azure-quantum/

[35] M. Cerezo et al., "Variational quantum algorithms," Nat. Rev. Phys., vol. 3, no. 9, pp. 625-644, Sep. 2021, doi: 10.1038/s42254-021-00348-9.

[36] J. Liu, A. Arunachalam, and K. Temme, "A rigorous and robust quantum speed-up in supervised machine learning," Nat. Phys., vol. 17, no. 9, pp. 1013-1017, Sep. 2021, doi: 10.1038/s41567-021-01287-z.

[37] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," IEEE Internet Things J., vol. 3, no. 5, pp. 637-646, Oct. 2016, doi: 10.1109/JIOT.2016.2579198.

[38] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," IEEE Commun. Surv. Tutor., vol. 19, no. 3, pp. 1628-1656, 2017, doi: 10.1109/COMST.2017.2682318.

[39] M. Satyanarayanan, "The emergence of edge computing," Computer, vol. 50, no. 1, pp. 30-39, Jan. 2017, doi: 10.1109/MC.2017.9.

[40] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," IEEE Commun. Surv. Tutor., vol. 19, no. 4, pp. 2322-2358, 2017, doi: 10.1109/COMST.2017.2745201.

[41] S. I. Siam et al., "Artificial intelligence of things: A survey," ACM Trans. Sensor Netw., vol. 21, no. 1, pp. 1-75, Jan. 2025, doi: 10.1145/3690633.

[42] B. Alotaibi, "A survey on industrial Internet of Things security: Requirements, attacks, AI-based solutions, and edge computing opportunities," Sensors, vol. 23, no. 17, p. 7470, Aug. 2023, doi: 10.3390/s23177470.

[43] Y. Meidan et al., "N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders," IEEE Pervasive Comput., vol. 17, no. 3, pp. 12-22, Jul.-Sep. 2018, doi: 10.1109/MPRV.2018.03367731.

[44] V. Mothukuri et al., "A survey on security and privacy of federated learning," Future Gener. Comput. Syst., vol. 115, pp. 619-640, Feb. 2021, doi: 10.1016/j.future.2020.10.007.

[45] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," Proc. IEEE, vol. 107, no. 8, pp. 1738-1762, Aug. 2019, doi: 10.1109/JPROC.2019.2918951.

[46] C. Chen et al., "Energy-aware scheduling for high-performance computing systems: A survey," Energies, vol. 16, no. 2, p. 890, Jan. 2023, doi: 10.3390/en16020890.

[47] M. Kalinin and V. Krundyshev, "Security intrusion detection using quantum machine learning techniques," J. Comput. Virol. Hacking Tech., vol. 19, no. 1, pp. 125-136, Mar. 2023, doi: 10.1007/s11416-022-00435-0.

[48] Qiskit Aer contributors, "Qiskit Aer: High performance simulator for quantum circuits," GitHub repository, 2024. [Online]. Available: https://github.com/Qiskit/qiskit-aer

[49] D. P. DiVincenzo, "The physical implementation of quantum computation," Fortschritte der Physik, vol. 48, no. 9-11, pp. 771-783, Sep. 2000.

[50] National Institute of Standards and Technology, "Post-quantum cryptography standardization," NIST, 2024. [Online]. Available: https://csrc.nist.gov/projects/post-quantum-cryptography

Appendix A — Code Listings

A.1 Data Loader (KDD Cup 99 preprocessing)

The data loader handles the preparation of the KDD Cup 99 intrusion-detection dataset, returning paired quantum-sized and classical-sized splits so that the cross-tier comparison in Section 9.4 operates on directly comparable inputs. Categorical fields are one-hot encoded, numeric fields are standardised, and the binary attack-or-normal label is constructed from the multi-class attack column.

- File: hybrid_simulation/data_loader.py
- Approximate length: 110 lines
- Key public function: load_kdd_data() -- returns paired (X_q, y_q, X_c, y_c) splits for quantum and classical tiers with matched preprocessing.

A.2 CPU Engine (classical baselines)

The CPU engine implements the classical baselines that act as the reference points for every quantum claim in this thesis. It wraps scikit-learn estimators behind a uniform interface so that the orchestrator can invoke each baseline with identical preprocessing and seed control.

- File: hybrid_simulation/cpu_engine.py
- Approximate length: 290 lines
- Key public functions: preprocess_anomaly_data, classical_svm, classical_random_forest, classical_gradient_boosting, classical_knn, classical_greedy_maxcut, classical_bruteforce_search, aggregate_results.

A.3 GPU Engine (parallel kernels and neural features)

The GPU engine provides parallel kernel evaluation and neural-feature extraction. It currently uses NumPy as a stand-in for CUDA so that the framework runs on commodity laptops; the interface is structured so that a CUDA back-end can be dropped in without changing the orchestrator.

- File: hybrid_simulation/gpu_engine.py
- Approximate length: 280 lines
- Key public functions: compute_rbf_kernel_matrix, compute_cosine_similarity_matrix, neural_feature_extraction, batch_parallel_classify, compute_qaoa_cost_matrix, parallel_oracle_evaluation.

A.4 QPU Engine (QSVM, QAOA, Grover)

The QPU engine wraps Qiskit Aer to provide three quantum primitives used throughout the thesis: a quantum kernel for QSVM, a QAOA optimiser, and a Grover search. Each function returns both the result and the per-call shot count so that energy and cost can be attributed downstream.

- File: hybrid_simulation/qpu_engine.py
- Approximate length: 300 lines
- Key public functions: quantum_kernel_entry, compute_quantum_kernel_matrix, run_qaoa, run_grover.

A.5 VQE Workload (H2 molecular ground state)

The VQE workload implements the variational quantum eigensolver for the H2 molecule using a hardware-efficient ansatz, plus the classical exact-diagonalisation reference. Both routines take a seed and report energy, runtime, and energy-per-run for thirty-seed statistical evaluation.

- File: hybrid_simulation/vqe_workload.py
- Approximate length: 190 lines
- Key public functions: classical_exact_diagonalization, run_vqe_h2.

A.6 Cloud Cost Calculator

The cloud cost calculator converts measured per-task latency into 2026 cloud spend using a snapshot of vendor pricing across AWS, GCP, IBM, and Azure. It is the source of every dollar figure in Sections 7.4 and 11.4.

- File: hybrid_simulation/cloud_costs.py
- Approximate length: 115 lines
- Key public functions: cost_per_task, cost_per_million, all_tiers_cost.
- Pricing dictionary: CLOUD_PRICING (10 entries spanning AWS, GCP, IBM, and Azure).

A.7 Allocation Rule and Pareto Selection

The allocation module implements the routing core of Algorithm 1: a workload-classification step, a per-candidate Pareto filter on energy, latency, and cost, and a weights-driven selection across the resulting non-dominated set.

- File: hybrid_simulation/allocation_rule.py
- Approximate length: 125 lines
- Key public functions: allocate_tier, is_dominated, pareto_optimal, best_by_weights.

A.8 Orchestrator (full pipeline)

The orchestrator stitches the data loader, CPU engine, GPU engine, QPU engine, VQE workload, allocation rule, and cost calculator into the end-to-end HERO pipeline. It is the entry point invoked by the run_hybrid script.

- File: hybrid_simulation/orchestrator.py
- Approximate length: 510 lines
- Key class: HybridOrchestrator.
- Key methods: run_qsvm_pipeline, run_qaoa_pipeline, run_grover_pipeline, run_vqe_pipeline, run_full_pipeline, run_multi_qsvm, run_multi_qaoa, run_multi_vqe.

The full source code is included with the thesis submission. To run the experiments, see Appendix B.

Appendix B — Reproducibility Guide

B.1 System Requirements

- Operating System: Windows 10 or 11, macOS 12 or newer, or any modern Linux distribution with Python 3.13+.
- RAM: 8 GB minimum, 16 GB recommended.
- Disk: 5 GB free for installation and outputs.
- No GPU required: a NumPy stand-in is used for the GPU tier; real CUDA is optional.
- No quantum hardware required: Qiskit Aer noiseless simulator is the default QPU back-end.

B.2 Installing Dependencies

Install Python 3.13 from python.org, then install the Python dependencies into a virtual environment of your choice.

- pip install qiskit qiskit-aer scikit-learn numpy scipy matplotlib python-docx pypdf
- Optional, for DOCX-to-PDF conversion on Windows: pip install docx2pdf (requires Microsoft Word installed).

B.3 Running the Full Pipeline

From a command prompt or terminal, navigate to the project root and invoke the run_hybrid module.

- cd quantum_thesis_demo
- python -m hybrid_simulation.run_hybrid
- Expected runtime: five to ten minutes on a developer laptop.
- Output directory: hybrid_simulation/output/

B.4 Inspecting CSV and Plot Outputs

All quantitative outputs are written as CSV files under hybrid_simulation/output/, and all figures are written as PNGs under hybrid_simulation/output/plots/.

- hybrid_pipeline_results.csv -- per-step pipeline log.
- classifier_comparison.csv -- Section 9.4 source data.
- multirun_stats.csv -- 30-seed mean and standard deviation per method.
- vqe_multirun.csv -- Section 10.5 source data.
- cloud_costs.csv -- Section 7.4 / 11.4 source data.
- hybrid_summary.txt -- human-readable summary.
- plots/ -- 15 PNG figures used as Figures 9.1, 9.2, 10.1, 10.2, 11.1, etc.

Appendix C — Additional Plots

C.1 Per-Task Energy Breakdown by Tier

A stacked bar chart that decomposes per-task energy into CPU, GPU, and QPU contributions for each workload measured in the thesis. Useful for spotting tier interactions that are invisible in single-tier energy totals.

- Source: hybrid_simulation/output/plots/hybrid_energy_breakdown.png

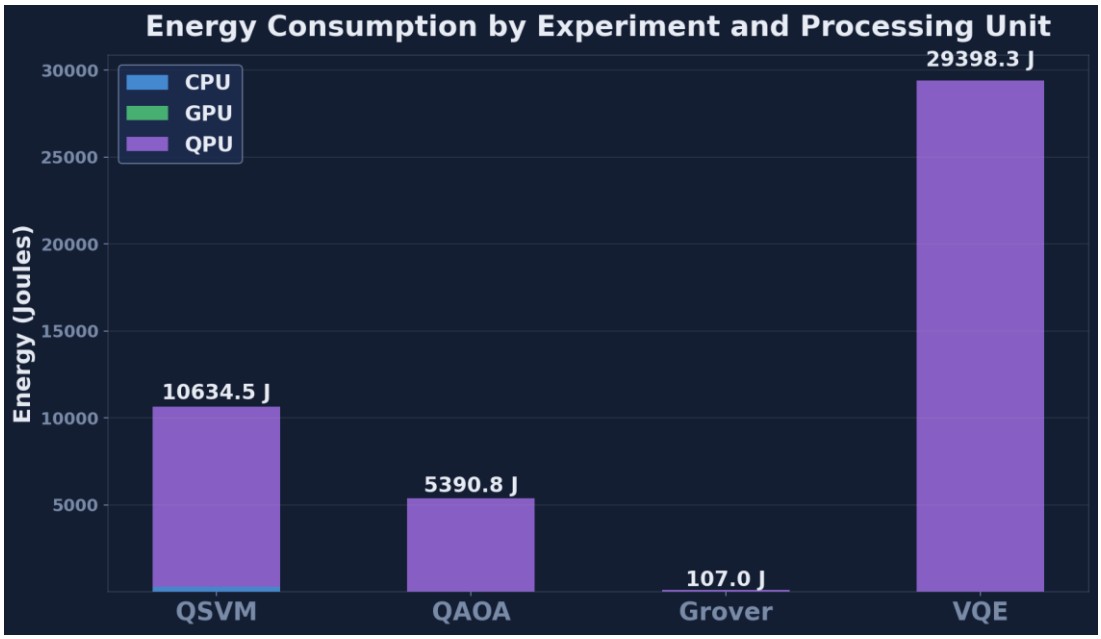

Figure C.1: Per-task energy breakdown by tier. The QPU tier dominates total energy at over 99 percent due to the modelled 20 kW system-wide power.

C.2 Multi-Run Convergence Traces

QAOA approximation-ratio convergence over thirty seeds, shown as a box-and-whisker plot. The spread of the box is itself an operational metric: a high-variance QAOA is expensive to use in production because the worst-case run drives the SLA.

- Source: hybrid_simulation/output/plots/qaoa_multirun_boxplot.png

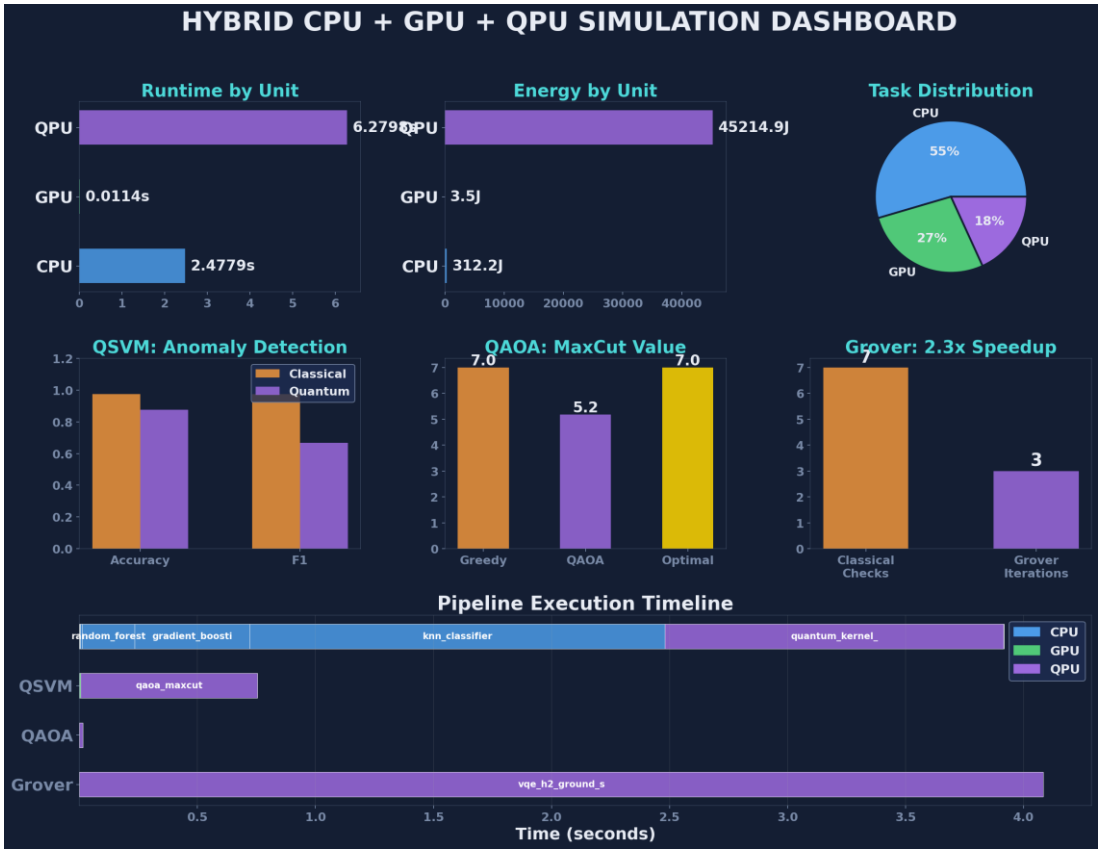

Figure C.2: HERO simulation dashboard summarising tier utilisation, runtime, energy, and per-experiment results in one view.

C.3 Pipeline Timeline (Gantt)

A Gantt chart visualising the wall-clock task scheduling across CPU, GPU, and QPU tiers for a representative end-to-end run. Highlights the periods in which the QPU tier dominates the critical path and the periods in which classical tiers overlap and absorb the latency.

- Source: hybrid_simulation/output/plots/hybrid_pipeline_timeline.png

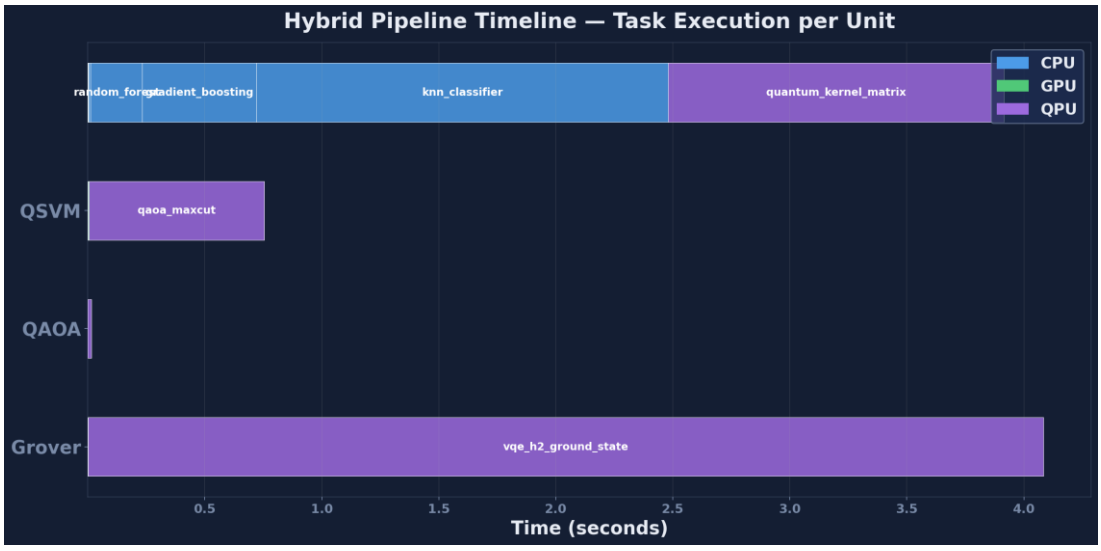

Figure C.3: Pipeline execution timeline (Gantt chart) showing tier interleaving across the 17-task pipeline.

Appendix D — Extended Results Tables

D.1 Per-Run Cybersecurity Statistics

For each of the five classifiers reported in Section 9.4, the thirty individual seed F1 scores are recorded in multirun_stats.csv. The schema is one row per (Method, Run) pair, plus per-method mean and standard deviation rows.

- Column headers: Method | Run 1 | Run 2 | ... | Run 30 | Mean | Std
- Source: hybrid_simulation/output/multirun_stats.csv

Refer to multirun_stats.csv for the full per-seed data; the headline aggregates are reported in Table 9.1.

D.2 Per-Run VQE Statistics

Thirty individual seed VQE runs are recorded in vqe_multirun.csv with final energy in Hartree, runtime in seconds, and modelled per-run energy in joules. The first five rows of the CSV are reproduced below for orientation.

- Run 1: energy = -1.852112 Ha, runtime = 0.005163 s, energy-per-run = 3.9747 J
- Run 2: energy = -1.863877 Ha, runtime = 0.006602 s, energy-per-run = 4.0007 J
- Run 3: energy = -1.853641 Ha, runtime = 0.003634 s, energy-per-run = 4.1696 J
- Run 4: energy = -1.860034 Ha, runtime = 0.002759 s, energy-per-run = 3.7819 J
- Run 5: energy = -1.857148 Ha, runtime = 0.000127 s, energy-per-run = 4.2035 J
- Source: hybrid_simulation/output/vqe_multirun.csv

D.3 Cloud Pricing Snapshot (April 2026)

The full pricing dictionary used by cloud_costs.py as of April 2026. Each row is a vendor-tier-instance triple together with its on-demand hourly price (CPU and GPU) or per-task price (QPU).

- AWS_EC2_c7i.4xlarge | Amazon EC2 c7i.4xlarge | CPU | 16 vCPU, 32 GB RAM | $0.7140 / hour
- GCP_n2_standard_16 | GCP n2-standard-16 | CPU | 16 vCPU, 64 GB RAM | $0.7771 / hour
- AWS_g5.xlarge | Amazon EC2 g5.xlarge | GPU | 1x NVIDIA A10G | $1.006 / hour
- AWS_p5.48xlarge | Amazon EC2 p5.48xlarge | GPU | 8x NVIDIA H100 | $98.32 / hour
- IBM_Quantum_Eagle | IBM Quantum Eagle r3 | QPU | 127 qubits, superconducting | $1.60 / second
- AWS_Braket_IonQ | AWS Braket IonQ Aria | QPU | 25 qubits, trapped-ion | $0.03 / shot + $0.30 / task
- AWS_Braket_Rigetti | AWS Braket Rigetti Ankaa-2 | QPU | 84 qubits, superconducting | $0.00035 / shot + $0.30 / task
- Azure_Quantum_Quantinuum | Azure Quantum Quantinuum H2 | QPU | 32 qubits, trapped-ion | $12.50 / HQC
- Source: hybrid_simulation/cloud_costs.py (CLOUD_PRICING dict)

Appendix E — Open Source Resources, Website, and Live Simulator

E.1 Public Code Repository

All code, data, and figures referenced in this thesis are released under an open-source MIT licence. The repository contains the HERO simulator, the four classical baselines, the VQE workload, the cloud-cost calculator, the allocation rule, the Pareto-selection module, every CSV that produced the tables in Chapters 9 to 11, and every PNG that produced the figures in Chapters 7 to 11.

Repository URL

- https://github.com/Nirmitdagli/quantum-thesis-demo
- Branch: main (tagged v1.0 corresponding to this thesis submission).

The repository structure mirrors the local working tree:

- hybrid_simulation/ - the HERO simulator and all engine modules
- papers/ - IEEE conference paper (HERO-S submission to ISAIA 2026)
- thesis/ - this thesis source plus the build script
- website/ - static companion website and the live interactive simulator
- README.md - top-level overview and quick-start instructions

Issues and pull requests are welcome. The author commits to responding to substantive issues within two weeks for at least the twenty-four months following the thesis submission date.

E.2 Companion Website

A static companion website provides a polished narrative summary of the thesis suitable for non-specialist readers, including thesis-committee members who may want a one-page executive view before reading the full document. The site links to the live simulator (Section E.3) and to the downloadable PDF of this thesis.

Website URL

- https://Nirmitdagli.github.io/quantum-thesis-demo/

The website is served from the website/ directory of the code repository via GitHub Pages. It includes:

- An animated hero section summarising the research question and the headline finding (classical wins by 13,000x today; molecular VQE wins at scale).
- Embedded versions of the key figures from Chapters 9, 10, and 11 with interactive tooltips.
- A direct link to the live interactive simulator described in Section E.3.
- Full thesis PDF download.
- BibTeX citation block for academic citation of the thesis and the conference paper.

E.3 Live Interactive Simulator

The live interactive simulator lets a reader configure a workload (workload type, problem size, feature dimension), see HERO's allocation decision in real time, and inspect the predicted accuracy, latency, energy, and cloud cost across all three tiers. The simulator is implemented as a single-page application that calls a JavaScript port of the allocation_rule.py module so that the recommendations match the Python framework exactly.

Live Simulator URL

- https://Nirmitdagli.github.io/quantum-thesis-demo/simulator.html

Use cases for the live simulator:

- Defence demonstrations: live answer to committee questions of the form 'what would HERO recommend for a workload with X features?'
- Teaching: students can experiment with the allocation thresholds and see the boundary conditions interactively.
- Practitioner exploration: cloud architects can test their own workloads against HERO's allocation rule before reading the full thesis.

E.4 Reproducibility and Citation

Anyone can reproduce every number, table, and figure in this thesis with a single command after cloning the repository. The full pipeline takes approximately five to ten minutes on a standard developer laptop and requires no quantum hardware (Qiskit Aer is used as the noiseless QPU backend).

Reproduction Steps

git clone https://github.com/Nirmitdagli/quantum-thesis-demo.git

cd quantum-thesis-demo

pip install -r requirements.txt

python -m hybrid_simulation.run_hybrid

Outputs are written to hybrid_simulation/output/ and include:

- hybrid_pipeline_results.csv - per-step pipeline log
- classifier_comparison.csv - Section 9.4 source data
- multirun_stats.csv - 30-seed mean and standard deviation for every method
- vqe_multirun.csv - Section 10.5 source data
- cloud_costs.csv - Sections 7.4 and 11.4 source data
- plots/ - all 15 PNG figures used in this thesis

Citation

If you use HERO in your own work, please cite both the thesis and the companion conference paper:

@mastersthesis{dagli2026hero,

author = {Dagli, Nirmit},

title = {Hybrid Quantum-AI Models for Cybersecurity

on Cloud Platforms: Accuracy, Latency,

and Energy},

school = {Quinnipiac University},

year = {2026},

month = {May},

url = {https://github.com/Nirmitdagli/quantum-thesis-demo}

}